WYTASOFT
Unlock the power of your data

DATA SCIENCE · NIVEAU 1

Python initiation au traitement de données

Du premier script au premier graphique : lire, nettoyer, calculer et tracer des données avec Python.

DURÉE	HORAIRES	NIVEAU	FORMATEUR
3 jours · 21 heures	09h00 – 17h30	Débutant	Mehdi Tajmouati

TOUR DE TABLE

Qui est dans la salle

Trois questions, une minute chacun :

1. Votre métier : vous **développez**, vous **analysez** des données, vous les **produisez** (mesures, fichiers, bases) ?
2. Votre programmation aujourd'hui : Excel et ses formules, VBA, R, SQL, un autre langage, rien de tout cela ?
3. Quelle tâche répétitive aimeriez-vous ne plus faire à la main ?

Pourquoi la troisième question

On vient rarement à Python par curiosité. On y vient parce qu'un classeur Excel est devenu ingérable, qu'un export doit être retraité chaque semaine, ou qu'un collègue l'a fait en dix lignes. La tâche citée dit à quoi servira la formation.

OBJECTIFS

Ce que vous saurez faire dans trois jours

1. **Écrire et organiser un programme Python**

   Types, boucles, tests, fonctions, modules : de quoi automatiser une tâche, et éviter les pièges classiques du langage.

2. **Lire des données là où elles sont**

   Fichiers texte, journaux bruts, bases relationnelles : les extraire avec la bibliothèque standard, les nettoyer, produire un CSV propre.

3. **Travailler dans un environnement maîtrisé**

   Anaconda, environnements virtuels, notebooks Jupyter : un projet qui tourne aussi chez votre collègue.

4. **Analyser et tracer avec la SciPy Stack**

   NumPy, pandas, Matplotlib et SciPy : statistiques, tableaux croisés, graphiques, carte, transformée de Fourier.

DÉROULÉ

Les trois jours d'un coup d'œil

	MATIN · 09H00 – 12H30	APRÈS-MIDI · 13H30 – 17H30
Jour 1	**Module 1** · Initiation au langage Python Anaconda, l'IDE, les erreurs, les types	**Module 1** (suite) · boucles et tests, fonctions, modules, pièges TP 1a : algorithmique · TP 1b : un module à soi, les dates
Jour 2	**Module 2** · Compléments sur le langage TP 2a : une classe, un fichier · TP 2b : journal + base → `mesures.csv`	**Module 3** · L'écosystème scientifique TP 3a : l'environnement · TP 3b : Jupyter et formats
Jour 3	**Module 4** · La SciPy Stack : NumPy, Matplotlib TP 4a : traiter une image avec NumPy	**Module 4** (suite) · pandas, SciPy TP 4b, 4c : statistiques, carte, Fourier · **bilan**

> Le fil est **progressif** : le CSV produit au TP 2b est celui qu'on ouvre dans Jupyter au TP 3b, puis qu'on analyse et qu'on trace aux TP 4a, 4b et 4c.

MODE D'EMPLOI

Comment lire ces supports

Base	Le socle du programme. Vu par tout le monde, c'est ce sur quoi vous serez à l'aise en sortant.
Avancé	Un approfondissement, hors programme contractuel. Traité selon le rythme du groupe.
Si le temps le permet	Une slide qui approfondit un sujet **déjà livré ailleurs** : la sauter ne retire rien au programme.

Le support vous est remis

En PDF, en fin de session, avec les fichiers de tous les TP et leurs corrigés. Vous n'avez rien à recopier : écoutez, essayez, et notez ce qui vous concerne.

POURQUOI PYTHON

Un langage simple, un écosystème qui fait le travail

Le même calcul, deux façons

```
releves = [41.2, 38.5, 55.0, 47.3]

total = 0
for valeur in releves:
    total = total + valeur
print(total / len(releves))
# 45.5

import statistics
print(statistics.mean(releves))
# 45.5
```

- **Lisible** : l'indentation fait partie de la syntaxe, le code ressemble à ce qu'il fait.
- **Interprété** : on tape une ligne, on voit le résultat. Idéal pour explorer des données.
- **Gratuit et ouvert**, sous Windows, macOS et Linux.
- **Un écosystème** : pandas, NumPy, Matplotlib… L'essentiel du travail est déjà écrit, il reste à l'assembler.

> Python n'est pas le langage le plus rapide. Il est rapide **à écrire**, et ses bibliothèques scientifiques font le calcul lourd en C.

FIL ROUGE

Air'Cité, le jeu de données des trois jours

Un réseau **fictif** de 12 capteurs de qualité de l'air, répartis dans 4 quartiers d'une ville, qui mesurent chaque heure.

`capteurs.db`	Une base SQLite : identifiant, nom, quartier, position, date de mise en service
`journal-capteurs.log`	Le journal brut, tel que les capteurs l'émettent : une ligne de texte par mesure ou par incident
`mesures.csv`	Le fichier propre que **vous** produirez au TP 2b : horodatage, capteur, quartier, NO_2, PM_{10}, température

```
2026-03-14T08:00:00 INFO  AQ-07 no2=41.2 pm10=23.5 temp=11.8
2026-03-14T08:00:00 ERROR AQ-03 timeout
```

Un fil, quatre étapes

Algorithmes et dates aux TP 1a et 1b, une classe puis l'extraction du journal aux TP 2a et 2b, l'environnement et Jupyter aux TP 3a et 3b, statistiques, carte et cycle quotidien du trafic aux TP 4a à 4c.

Volontairement imparfait

Lignes tronquées, valeurs manquantes, incidents mêlés aux mesures : comme un vrai journal.

CADRAGE

Ce que cette formation n'est pas

Pas un cours de génie logiciel

On écrit des scripts et des notebooks propres. Les tests automatisés, l'empaquetage et l'architecture d'une application sont évoqués, pas enseignés.

Pas un cours de statistiques

On calcule des moyennes, des corrélations, un test de Student. On dit ce qu'ils mesurent, pas leur théorie.

Pas un cours de machine learning

scikit-learn est présenté dans le panorama du module 3. L'apprentissage automatique est la suite logique, pas le sujet.

Ce que vous saurez faire en sortant : **prendre un fichier qu'on vous donne, et en tirer un chiffre et un graphique**, avec un script que vous pourrez relancer le mois prochain.

RÈGLES DU JEU

Comment on travaille

Pendant les exposés

- **Coupez-moi.** Une question posée au moment où elle se pose vaut mieux qu'une question gardée pour la fin.
- Le code des slides est fait pour être **tapé et exécuté**, dans l'interpréteur ou l'IDE ouvert à côté.
- Aucune question n'est trop simple. « Pourquoi des deux-points à la fin du `if` ? » est une question de la salle, pas une faute.

Pendant les TP

- Tout est dans le dossier `tp/` remis en début de session : les énoncés, les fichiers de départ, les données.
- Chaque TP a un **énoncé**, un **point de départ** et un **corrigé** projeté en fin de TP.
- Une erreur Python n'est pas un échec : c'est un message à lire. On apprend à les lire ensemble dès le premier jour.
- Les étapes **bonus** sont pour ceux qui ont fini, pas pour tout le monde.

AVANT DE COMMENCER

Trois questions pour situer la salle

Q1 Vous devez additionner une colonne dans 200 fichiers Excel. Quelle est la partie difficile ?

Pas l'addition : la répétition. Ouvrir, trouver la colonne, recommencer. C'est ce qu'une boucle et une fonction automatisent.

Q2 Une cellule contient `2026-03-14T08:00:00`. Est-ce un texte ou une date ?

Les deux, selon qui la lit. Pour un programme, c'est un texte tant qu'on ne l'a pas converti : on ne peut ni la comparer à une autre date, ni lui ajouter une heure.

Q3 Un fichier contient 0,5 % de lignes abîmées. Les ignorer, les corriger ou tout arrêter ?

Ça dépend, mais jamais en silence : on les écarte **et on les compte**. Un traitement qui ne dit pas ce qu'il a rejeté n'est pas fiable.

MODULE 1

Initiation au langage Python

L'environnement de travail, les types, les boucles et les tests, les fonctions, les modules, et les pièges du langage.

JOUR 1 · MODULE 1

Initiation au langage Python

L'environnement, les types, les boucles, les fonctions, les modules, et les pièges qui coûtent une après-midi.

DURÉE
La journée · ≈ 4 h 35 d'exposé et d'exercices + 2 TP (1 h 30)

À LA FIN
Écrire et lancer un script Python qui tient debout

TP
1a algorithmique · 1b un module à soi, et les dates

FEUILLE DE ROUTE

Ce que couvre le module 1

1. **L'environnement**
   Anaconda, un IDE, script ou interactif, lire une erreur, le débogueur.
2. **Les types de données**
   Nombres, booléens, chaînes, listes, tuples, dictionnaires, et les exceptions.
3. **Les structures de contrôle**
   `if`, `for`, `while`, et les compréhensions.
4. **Les fonctions**
   Paramètres, valeurs par défaut, arguments variables, portée.
5. **Les librairies**
   Modules, paquets, `import`, la bibliothèque standard.
6. **Les pièges**
   Mutable, immuable, références, copies, et les dates avant le TP.

Le fil de la journée

Une ligne du journal des capteurs Air'Cité :

```
2026-03-14T08:00:00 INFO AQ-07 no2=41.2
```

Ce soir, vous savez la **découper**, la **convertir**, la **juger** et en tirer une date. C'est le début du TP 2b.

SÉQUENCE 1

L'environnement de développement

Où vit Python sur votre poste, et les deux façons de lui parler.

Un langage simple, un écosystème immense

- **Interprété** : pas de compilation, on écrit, on exécute, on voit.
- **Lisible** : l'indentation fait partie de la syntaxe, le code ressemble à du pseudo-code.
- **Typé dynamiquement** : une variable n'a pas de type, la valeur en a un.
- **Polyvalent** : scripts d'administration, web, automatisation, et surtout la donnée.

Le vrai atout pour la donnée

Le langage est lent, mais **NumPy, pandas, SciPy** sont écrits en C et en Fortran. Python sert de colle : on décrit le calcul en Python, il s'exécute en code compilé.

Python 3, et rien d'autre

Python 2 est arrêté depuis 2020. Tout ce qui est montré ici vise **Python 3.11 ou plus récent**.

Ce qu'il y a dans la boîte



Une distribution, c'est Python **plus** tout ce qu'il faut pour travailler la donnée, testé ensemble, installé en une fois.

Anaconda Prompt (sous Windows) : un terminal où `python` et `conda` sont trouvés.

> Le pourquoi d'une distribution et des environnements est le sujet du **module 3**. Aujourd'hui, on s'en sert.

DEUX FAÇONS DE TRAVAILLER BASE

L'interactif, ou le script

L'interpréteur interactif

```
>>> 41.2 + 38.7
79.9
>>> "AQ-07".lower()
'aq-07'
>>> exit()
```

On tape `python`, l'invite `>>>` attend. Chaque ligne est exécutée, **sa valeur s'affiche**. Idéal pour essayer, rien n'est gardé.

Le script

```
# releve.py
no2 = 41.2
print("NO2 :", no2)
```

```
> python releve.py
NO2 : 41.2
```

Un fichier `.py`, exécuté du début à la fin. Rien ne s'affiche **sans** `print`. C'est ce qu'on garde, relance et partage.

Un éditeur qui sait exécuter

Spyder

Livré avec Anaconda. Éditeur, console IPython et **explorateur de variables** dans la même fenêtre. Pensé pour le calcul scientifique, proche de MATLAB ou RStudio.
Celui de la formation.

VS Code

Gratuit, léger, généraliste. L'extension Python ajoute l'exécution, le débogueur, les notebooks. Le plus répandu aujourd'hui.

PyCharm

L'IDE Python de JetBrains. Très complet sur la refactorisation et les projets. Une édition gratuite suffit pour ce qu'on fait ici.

> Ce qui compte : un IDE qui utilise **le bon interpréteur**. Dans VS Code comme dans PyCharm, on choisit l'interpréteur Anaconda une fois pour toutes.

À VOUS **BASE**

Un premier script, dans Spyder

```
capteur = "AQ-07"
no2 = 41.2
seuil = 40

print("Capteur", capteur)
print("NO2 :", no2, "µg/m³")
print("Au-dessus du seuil ?", no2 > seuil)
```

```
Capteur AQ-07
NO2 : 41.2 µg/m³
Au-dessus du seuil ? True
```

1. **Créer `premier.py`**

   Fichier, nouveau, enregistrer.

2. **Taper, puis F5**

   La sortie s'affiche dans la console.

3. **Regarder l'explorateur de variables**

   Trois variables, et leur type.

4. **Changer `no2`, relancer**

   Et taper `no2 * 2` dans la console.

5. **Bonus : `sep` et `input`**

   `print(capteur, no2, sep=";")` écrit `AQ-07;41.2`.

   `no2 = float(input("NO2 ? "))` demande la valeur.

LIRE UNE ERREUR BASE

Un message d'erreur se lit en partant du bas

```python
def lire_no2(morceau):
    nom, valeur = morceau.split("=")
    return float(valeur)

def moyenne_no2(morceaux):
    total = 0
    for m in morceaux:
        total += lire_no2(m)
    return total / len(morceaux)

print(moyenne_no2(["no2=41.2", "no2=abc"]))
```

```
Traceback (most recent call last):
  File "moyenne.py", line 11, in <module>
    print(moyenne_no2(["no2=41.2", "no2=abc"]))
  File "moyenne.py", line 8, in moyenne_no2
    total += lire_no2(m)
  File "moyenne.py", line 3, in lire_no2
    return float(valeur)
ValueError: could not convert string to float: 'abc'
```

1. **La dernière ligne**

   Le **type** d'erreur et le message : ici, `'abc'` n'est pas un nombre.

2. **Le dernier `File`**

   Où l'erreur a eu lieu : ligne 3, dans `lire_no2`.

3. **En remontant**

   Qui a appelé qui : le chemin jusqu'à votre ligne 11.

`NameError`	Nom inconnu : faute de frappe, variable pas encore définie
`TypeError`	Opération entre types incompatibles : `"AQ-" + 7`
`ValueError`	Bon type, mauvaise valeur : `int("41.2")`
`IndexError`	Position hors de la liste
`KeyError`	Clé absente du dictionnaire
`IndentationError`	Bloc mal aligné

Arrêter le programme, et regarder dedans

1. **Poser un point d'arrêt**

   Clic dans la marge, à gauche du numéro de ligne : un point rouge.

2. **Lancer en mode débogage**

   Le programme s'exécute normalement, puis **s'arrête** sur la ligne marquée, avant de l'exécuter.

3. **Regarder les variables**

   Leur valeur à cet instant : explorateur de variables dans Spyder, panneau *Variables* dans VS Code.

4. **Avancer pas à pas**

   Ligne suivante, ou entrer dans la fonction appelée, ou continuer jusqu'au prochain point d'arrêt.

ACTION	SPYDER	VS CODE
Point d'arrêt	`F12`	`F9`
Déboguer	`Ctrl+F5`	`F5`
Ligne suivante	`Ctrl+F10`	`F10`
Entrer dans l'appel	`Ctrl+F11`	`F11`
Continuer	`Ctrl+F12`	`F5`

> Sans IDE : la fonction `breakpoint()`, placée dans le code, arrête le programme au même endroit, dans le terminal.

SÉQUENCE 2

Les principaux types de données

Nombres, booléens, chaînes, puis les trois conteneurs qu'on utilise tous les jours.

Une variable est une étiquette, la valeur a un type

```
x = 41.2
type(x).__name__
# 'float'

x = "41.2"
type(x).__name__
# 'str'
```

```
x = 41.2
y = "41.2"
x + y
# TypeError: unsupported operand type(s)
#   for +: 'float' and 'str'
x + float(y)
# 82.4
```

- **Typage dynamique** : la même variable peut désigner un nombre, puis une chaîne. On ne déclare rien.
- **Typage fort** : Python ne convertit pas en douce. Un nombre plus une chaîne, c'est une erreur, pas `"41.241.2"`.
- Noms en `minuscules_avec_soulignés`, sans accent par convention.

> Tout ce qu'on lit dans un fichier arrive en **chaîne**. Convertir est la première chose qu'on fait.

Entiers, flottants, et leurs opérateurs

```
7 / 2      # 3.5   division
7 // 2     1
7 % 2      # 1     reste
2 ** 10    # 1024  puissance
```

```
int("42") + 1         # 43
float("41.2")         # 41.2
str(42) + "!"         # '42!'
int(41.9)             # 41, tronqué
round(41.256, 1)      # 41.3
1_000_000             # 1000000
```

TYPE	EXEMPLE	REMARQUE
`int`	`42`	Taille illimitée
`float`	`41.2`	Point, pas virgule
`complex`	`2+3j`	Rare, sauf en signal

> `round(41.5)` donne `42`, et `round(42.5)` donne… `42`. Python arrondit au **pair le plus proche**, la règle des banquiers.

Vrai, faux, et ce que Python considère comme vide

```
no2 = 41.2
no2 > 40                # True
no2 == 41.2             # True, deux signes égal
40 < no2 ≤ 50           # True, comparaison chaînée

True and False          # False
True or False           # True
not True                # False
```

La « véracité » des valeurs

```
bool(0)      # False
bool("")     # False
bool([])     # False
bool("0")    # True, chaîne non vide
```

Zéro, la chaîne vide, la liste vide, `None` valent faux. Le reste vaut vrai.

> `if releves:` se lit « s'il y a des relevés ».

Indexer, découper, interroger

```
ligne = "2026-03-14T08:00:00 INFO  AQ-07 no2=41.2"

len(ligne)          # 40
ligne[:10]          # '2026-03-14'
ligne[-8:]          # 'no2=41.2'
ligne.split()
# ['2026-03-14T08:00:00', 'INFO',
#  'AQ-07', 'no2=41.2']
"no2=41.2".split("=")
# ['no2', '41.2']
```

```
"AQ-07".lower()             # 'aq-07'
"  INFO  ".strip()          # 'INFO'
"07" in "AQ-07"             # True
"AQ-07".startswith("AQ")    # True
"-".join(["AQ", "07"])      # 'AQ-07'
"a=1".replace("=", " : ")   # 'a : 1'
```

- On compte **à partir de 0**, et `-1` est le dernier.
- `[début:fin]` : la fin est **exclue**.
- Une chaîne est **immuable** : chaque méthode renvoie une nouvelle chaîne.

CHAÎNES FORMATÉES BASE

Les f-strings, pour écrire un résultat lisible

```python
capteur = "AQ-07"
no2 = 41.2

print(f"{capteur} : {no2:.0f} µg/m³")
# AQ-07 : 41 µg/m³

[f"AQ-{n:02d}" for n in [1, 7, 12]]
# ['AQ-01', 'AQ-07', 'AQ-12']
```

FORMAT	EFFET
`{x:.1f}`	Une décimale
`{n:02d}`	Entier sur deux chiffres, complété de zéros
`{x:>8}`	Aligné à droite sur 8 caractères
`{x=}`	Affiche le nom et la valeur, pour déboguer

Le `f` devant les guillemets

Sans lui, les accolades sont du texte. Avec lui, ce qu'elles contiennent est **évalué** : une variable, un calcul, un appel.

Les anciennes façons

`"%s : %d" % (a, b)` et `"{} : {}".format(a, b)` existent dans le code ancien. Les f-strings les remplacent.

La liste : une séquence ordonnée et modifiable

```
releves = [41.2, 38.7, 52.1]
releves.append(47.0)
releves
# [41.2, 38.7, 52.1, 47.0]

releves[0], releves[-1]   # (41.2, 47.0)
releves[1:3]              # [38.7, 52.1]

len(releves)              # 4
max(releves)              # 52.1
sum(releves) / len(releves)
# 44.75

releves.sort()
releves
# [38.7, 41.2, 47.0, 52.1]
```

MÉTHODE	EFFET
`append(x)`	Ajoute à la fin
`insert(i, x)`	Insère à la position `i`
`pop()`	Retire et renvoie le dernier
`remove(x)`	Retire la première occurrence
`sort()`	Trie **sur place**, renvoie `None`
`sorted(l)`	Renvoie une **nouvelle** liste triée

> `releves = releves.sort()` remplace vos données par `None`.

Le tuple : une séquence figée

```
position = (48.85, 2.35)
lat, lon = position        # déballage
lat
# 48.85

position[0] = 0
# TypeError: 'tuple' object does not
#   support item assignment
```

```
a, b = b, a                # échange sans variable temporaire
seul = (42,)               # la virgule fait le tuple
```

Quand choisir un tuple

- Un **enregistrement** de taille fixe : une position, une paire (capteur, valeur).
- Plusieurs valeurs **renvoyées** par une fonction.
- Une **clé** de dictionnaire composée : `("AQ-07", "2026-03-14")`.

> Liste : beaucoup d'éléments **de même nature**. Tuple : peu d'éléments **de natures différentes**.

Le dictionnaire : retrouver une valeur par sa clé

```
capteur = {"id": "AQ-07", "quartier": "Port",
           "no2": 41.2}
capteur["quartier"]       # 'Port'
capteur["pm10"] = 23.5    # ajoute une clé

capteur["temp"]
# KeyError: 'temp'
capteur.get("temp")           # None
capteur.get("temp", "n/a")    # 'n/a'
```

```
for cle, val in capteur.items():
    print(cle, "→", val)
# id → AQ-07
# quartier → Port
# no2 → 41.2
# pm10 → 23.5
```

- Des paires **clé : valeur**. La clé est unique et immuable : chaîne, nombre, tuple.
- Accès **immédiat** par la clé, quelle que soit la taille.
- L'ordre d'insertion est conservé.
- `keys()`, `values()`, `items()` pour parcourir.

Le type de la donnée

Une ligne de CSV, une réponse d'API en JSON, un enregistrement de base : en Python, tout cela devient naturellement un dictionnaire.

L'ensemble : des éléments uniques

```
ids = ["AQ-03", "AQ-07", "AQ-03", "AQ-12"]
sorted(set(ids))
# ['AQ-03', 'AQ-07', 'AQ-12']
len(set(ids))
# 3

{"AQ-03", "AQ-07"} - {"AQ-07"}
# {'AQ-03'}
```

- **Dédoublonner** une liste en une ligne.
- Tester l'appartenance aussi vite qu'avec un dictionnaire.
- Union `|`, intersection `&`, différence `-` : « les capteurs attendus qui n'ont rien envoyé ».

> Un ensemble **n'a pas d'ordre**. Pour l'afficher de façon stable, on le trie.

SYNTHÈSE BASE

Quel conteneur pour quel usage

TYPE	ÉCRITURE	ORDONNÉ	MODIFIABLE	DOUBLONS	USAGE TYPIQUE
`list`	`[1, 2]`	oui	oui	oui	Une série de relevés
`tuple`	`(1, 2)`	oui	non	oui	Une position, un enregistrement fixe
`dict`	`{"a": 1}`	ordre d'insertion	oui	clés uniques	Un capteur et ses attributs, des comptes
`set`	`{1, 2}`	non	oui	non	Des identifiants distincts
`str`	`"ab"`	oui	non	oui	Du texte, une ligne de journal

> Deux colonnes à retenir pour cet après-midi : **modifiable** ou non. Toute la séquence des pièges en découle.

À VOUS **BASE**

Décortiquer une ligne du journal

À partir de :

```
ligne = "2026-03-14T08:00:00 INFO  AQ-07 no2=41.2 pm10=23.5 temp=11.8"
```

1. **La date et l'heure**

   `'2026-03-14'` et `'08:00:00'`

2. **Le niveau et le capteur**

   `'INFO'`, `'AQ-07'`

3. **Un dictionnaire des mesures, en nombres**

   `{'no2': 41.2, 'pm10': 23.5, 'temp': 11.8}`

4. **Une phrase de résultat**

   `AQ-07 le 2026-03-14 à 08h : NO2 41.2`

Indices

`split()`, puis `split("=")` sur chaque morceau à partir du quatrième. `float()` pour convertir. Une f-string pour finir.

Pour ceux qui vont vite

Même chose avec `"2026-03-14T09:00:00 ERROR AQ-03 timeout"`. Que casse votre code ?

Rattraper une erreur au lieu de s'arrêter

```
lignes = ["no2=41.2", "no2=", "no2=abc", "no2=52.1"]
valeurs, rejets = [], 0

for ligne in lignes:
    try:
        v = float(ligne.split("=")[1])
    except ValueError:
        rejets += 1
    else:
        valeurs.append(v)

valeurs, rejets
# ([41.2, 52.1], 2)
```

```
quartiers = {"AQ-01": "Centre", "AQ-07": "Port"}
for c in ["AQ-07", "AQ-99"]:
    try:
        print(c, "→", quartiers[c])
    except KeyError as e:
        print("capteur inconnu :", e)
# AQ-07 → Port
# capteur inconnu : 'AQ-99'
```

`try`	Le code qui peut échouer, le plus court possible
`except Err`	Exécuté si **cette** erreur survient ; `as e` donne le message
`else`	Exécuté s'il n'y a **pas** eu d'erreur
`finally`	Exécuté dans tous les cas : fermer, journaliser

```
def lire_seuil(texte):
    seuil = float(texte)
    if seuil ≤ 0:
        raise ValueError(f"seuil invalide : {seuil}")
    return seuil

lire_seuil("-5")
# ValueError: seuil invalide : -5.0
```

> Jamais de `except:` nu : il avale aussi les fautes de frappe et le Ctrl+C. Nommer l'erreur attendue.

SÉQUENCE 3

Les structures de contrôle

Décider avec `if`, répéter avec `for` et `while`, et la compréhension qui les résume.

TESTS BASE

`if`, `elif`, `else`, et l'indentation qui fait le bloc

```python
def qualite(no2):
    if no2 < 40:
        return "bon"
    elif no2 < 100:
        return "moyen"
    else:
        return "mauvais"

qualite(41.2), qualite(12), qualite(230)
# ('moyen', 'bon', 'mauvais')
```

- Les deux-points `:` ouvrent un bloc.
- Le bloc, c'est l'**indentation** : quatre espaces, toujours les mêmes. Pas d'accolades, pas de `end`.
- Les conditions sont testées dans l'ordre, la **première vraie** gagne.

```python
statut = "alerte" if no2 > 200 else "normal"
```

> Mélanger tabulations et espaces donne une `IndentationError`. Les IDE convertissent : laissez-les faire.

BOUCLE `FOR` BASE

Parcourir n'importe quelle séquence

```
releves = [41.2, 38.7, 52.1, 47.0]

for valeur in releves:
    if valeur > 45:
        print("dépassement :", valeur)
# dépassement : 52.1
# dépassement : 47.0
```

```
list(range(5))           # [0, 1, 2, 3, 4]
list(range(1, 13, 4))    # [1, 5, 9]
```

Le `for` de Python parcourt **les éléments**, pas des indices :

- une liste : ses valeurs ;
- une chaîne : ses caractères ;
- un dictionnaire : ses **clés** ;
- un fichier ouvert : ses lignes (module 2).

> `range` sert quand on a vraiment besoin d'un compteur. C'est plus rare qu'on ne croit.

`enumerate` et `zip` : l'indice, et deux séquences à la fois

Numéroter en parcourant

```
for i, v in enumerate(releves, start=1):
    print(i, v)
# 1 41.2
# 2 38.7
# 3 52.1
# 4 47.0
```

Parcourir en parallèle

```
capteurs = ["AQ-01", "AQ-02", "AQ-03"]
no2 = [41.2, 38.7, 52.1]

for c, v in zip(capteurs, no2):
    print(c, v)
# AQ-01 41.2
# AQ-02 38.7
# AQ-03 52.1
```

> `zip` s'arrête à la **plus courte** des séquences, sans prévenir.

BOUCLE `WHILE` BASE

Répéter tant que, et sortir plus tôt

```
essais = 0
while essais < 3:
    essais += 1
    print("tentative", essais)
# tentative 1
# tentative 2
# tentative 3
```

Quand on ne sait pas d'avance **combien** de tours : relancer une connexion, converger vers une valeur.

```
for v in [41.2, None, 52.1]:
    if v is None:
        continue          # passe au suivant
    print(v)
# 41.2
# 52.1

for ligne in ["INFO ok", "ERROR timeout", "INFO ok"]:
    if ligne.startswith("ERROR"):
        print("arrêt sur :", ligne)
        break             # sort de la boucle
# arrêt sur : ERROR timeout
```

> Un `while` dont la condition ne devient jamais fausse tourne pour toujours. `Ctrl+C` l'interrompt.

COMPRÉHENSIONS BASE

Construire une liste en une ligne

La boucle

```
depassements = []
for v in releves:
    if v > 40:
        depassements.append(v)
```

La compréhension

```
depassements = [v for v in releves if v > 40]
# [41.2, 52.1, 47.0]
```

```
[f"AQ-{n:02d}" for n in range(1, 13)]
# ['AQ-01', 'AQ-02', ... , 'AQ-12']

{c: v for c, v in zip(capteurs, no2)}
# {'AQ-01': 41.2, 'AQ-02': 38.7, 'AQ-03': 52.1}
```

Se lit de gauche à droite : « **ce que je garde**, pour chaque élément, **si** la condition ». Existe aussi pour les dictionnaires et les ensembles.

Au-delà d'une ligne, revenir à la boucle.

SÉQUENCE 4

Les fonctions

Nommer un calcul, lui passer des paramètres, en recevoir un résultat.

CRÉER UNE FONCTION BASE

`def`, `return`, et la docstring

```python
def moyenne(valeurs):
    """Renvoie la moyenne d'une liste non vide."""
    return sum(valeurs) / len(valeurs)

moyenne([41.2, 38.7, 52.1])
# 44.0

help(moyenne)
# moyenne(valeurs)
#     Renvoie la moyenne d'une liste non vide.
```

- `def`, un nom, les paramètres entre parenthèses, deux-points, un bloc indenté.
- `return` renvoie une valeur et **sort** de la fonction.
- Sans `return`, la fonction renvoie `None`.
- La **docstring**, juste sous `def`, est la documentation que lisent `help()` et l'IDE.

> `print` affiche, `return` renvoie. Une fonction qui affiche son résultat sans le renvoyer ne sert à rien au code qui l'appelle.

PARAMÈTRES BASE

Valeurs par défaut et arguments nommés

```python
def seuil_depasse(valeur, seuil=40):
    return valeur > seuil

seuil_depasse(41.2)                  # True
seuil_depasse(41.2, seuil=50)        # False
seuil_depasse(seuil=50, valeur=60)   # True
```

```python
def mini_maxi(valeurs):
    return min(valeurs), max(valeurs)

bas, haut = mini_maxi([41.2, 38.7, 52.1])
# bas = 38.7, haut = 52.1
```

- Un paramètre **avec valeur par défaut** devient facultatif. Il se place après ceux qui n'en ont pas.
- À l'appel, on peut **nommer** les arguments : l'ordre ne compte plus, et le code se lit tout seul.
- Plusieurs valeurs de retour ? On renvoie un **tuple**, qu'on déballe.

> `seuil_depasse(41.2, seuil=50)` se lit mieux que `seuil_depasse(41.2, 50)`. Nommez tout ce qui n'est pas évident.

`*args` et `**kwargs`

Un nombre quelconque de valeurs

```python
def total(*valeurs):
    print(valeurs)
    return sum(valeurs)

total(1, 2, 3)
# (1, 2, 3)
# 6
```

Un nombre quelconque de noms

```python
def mesure(capteur, **valeurs):
    print(capteur, valeurs)

mesure("AQ-07", no2=41.2, pm10=23.5)
# AQ-07 {'no2': 41.2, 'pm10': 23.5}
```

- `*` ramasse les arguments positionnels en surplus dans un **tuple**.
- `**` ramasse les arguments nommés en surplus dans un **dictionnaire**.
- `args` et `kwargs` ne sont que des noms d'usage.

Dans l'autre sens, à l'appel

```python
releves = [41.2, 38.7, 52.1]
max(*releves)                  # 52.1

options = {"seuil": 50}
seuil_depasse(41.2, **options) # False
```

PORTÉE DES VARIABLES BASE

Ce qui naît dans une fonction y reste

```python
seuil = 40                  # globale, au niveau du module

def depasse(v):
    return v > seuil        # lue : trouvée dehors

def regler():
    seuil = 50              # une NOUVELLE variable locale
    return seuil

depasse(41.2)               # True
regler(), seuil             # (50, 40)
```

```python
compteur = 0
def compter():
    compteur += 1
compter()
# UnboundLocalError: cannot access local variable
#   'compteur' where it is not associated with a value
```

Python cherche un nom dans cet ordre, la règle **LEGB** :

1. **L**
   ocalLes paramètres et variables de la fonction.
2. **E**
   nglobantLa fonction qui contient la fonction, s'il y en a une.
3. **G**
   lobalLe niveau du module, le fichier.
4. **B**
   uilt-in `len`, `print`, `max`…

> Une fonction **lit** les variables du dehors, mais une affectation crée une variable **locale**. Pour sortir un résultat : `return`, pas `global`.

Dire quel type on attend, sans l'imposer

```python
def qualite(no2: float) -> str:
    if no2 < 40:
        return "bon"
    return "moyen"

def moyenne(valeurs: list[float]) -> float:
    return sum(valeurs) / len(valeurs)

moyenne.__annotations__
# {'valeurs': list[float], 'return': <class 'float'>}
```

```python
qualite("41.2")
# TypeError: '<' not supported between
#   instances of 'str' and 'int'
```

- `: float` après un paramètre, `-> str` après la parenthèse : le type attendu, le type renvoyé.
- **Ignorées à l'exécution** : Python ne vérifie rien, `qualite("41.2")` plante comme avant.
- Utiles à l'**IDE**, qui souligne l'appel fautif avant qu'on l'exécute, et au lecteur.
- Un outil comme **mypy** les vérifie sur tout un projet.

> On les croisera dans le code récent et dans la documentation des librairies. Savoir les **lire** suffit.

FONCTIONS EN ARGUMENT SI LE TEMPS LE PERMET

Passer une fonction : `key=` et `lambda`

```python
sorted(["AQ-12", "AQ-3", "AQ-07"], key=len)
# ['AQ-3', 'AQ-12', 'AQ-07']

paires = [("AQ-01", 41.2), ("AQ-02", 38.7)]
sorted(paires, key=lambda p: p[1])
# [('AQ-02', 38.7), ('AQ-01', 41.2)]
```

- Une fonction est une valeur comme une autre : on peut la passer à `sorted`, `max`, `min`.
- `lambda p: p[1]` est une fonction **sans nom**, d'une seule expression : « prends `p`, renvoie `p[1]` ».

> On retrouvera ce style avec pandas : `df["no2"].apply(qualite)` applique votre fonction à chaque valeur.

TRAVAUX PRATIQUES 1A

Algorithmique : des boucles, des tests, des fonctions

≈ 40 min · Individuel · Restitution 5 min · `tp/tp1a-algorithmique/`

1. **L'environnement**

   Anaconda Prompt : `python --version`. `depart.py` lancé depuis Spyder (`F5`), puis depuis le terminal.

2. **FizzBuzz et nombres premiers**

   `for`, `if/elif/else`, `%`, un `while` pour chercher un diviseur, une compréhension.

3. **La plus longue série**

   Le plus grand nombre de relevés consécutifs au-dessus d'un seuil : deux compteurs.

4. **Compter les niveaux du journal**

   Un dictionnaire niveau → nombre, d'abord sur six lignes, puis sur le vrai journal.

Les fichiers

`tp/tp1a-algorithmique/` :
`enonce.md` · `depart.py`
`corrige.py`, projeté en restitution

Un résultat pour se vérifier

Série `[38.7, 41.2, 45.0, 52.1, 39.0, 47.0, 48.3]`, seuil 40 : la plus longue série de dépassements fait **3**.

Bonus

Moyenne glissante sur 3 relevés : `[41.6, 46.1, 45.4, 46.0, 44.8]`.

SÉQUENCE 5

Créer et utiliser des librairies

Un fichier devient un module, un dossier devient un paquet, et la bibliothèque standard est déjà là.

Un fichier est un module, un dossier est un paquet

```
projet/
├── analyse.py
└── aircite/
    ├── __init__.py
    └── qualite.py
```

```python
# aircite/qualite.py
"""Seuils de qualité de l'air."""
SEUIL_NO2 = 40

def qualite(no2):
    if no2 < SEUIL_NO2:
        return "bon"
    elif no2 < 100:
        return "moyen"
    return "mauvais"
```

```python
# analyse.py
from aircite.qualite import qualite, SEUIL_NO2
import aircite.qualite as q

print(qualite(41.2), SEUIL_NO2)
print(q.__name__)
```

```
> python analyse.py
moyen 40
aircite.qualite
```

`import math`	`math.sqrt(2)`
`import numpy as np`	Un alias, par convention
`from math import sqrt`	`sqrt(2)` directement
`from math import *`	À éviter

`if __name__ == "__main__":`

```python
# à la fin de aircite/qualite.py
if __name__ == "__main__":
    print("test :", qualite(41.2))
```

```
> python aircite/qualite.py
test : moyen

> python analyse.py
moyen 40
aircite.qualite
```

- Chaque module a une variable `__name__`.
- Lancé directement, elle vaut `"__main__"` : le bloc s'exécute.
- Importé, elle vaut le nom du module : le bloc est **ignoré**.

> Le même fichier sert de **librairie** pour les autres et de **script** pour ses propres essais.

BIBLIOTHÈQUE STANDARD BASE

Les piles fournies : quelques modules à connaître

```python
import math, statistics, random
from collections import Counter
from pathlib import Path

round(math.pi, 3)                # 3.142
statistics.median([41.2, 38.7, 52.1, 47.0])
# 44.1
Counter(["INFO", "ERROR", "INFO",
         "INFO", "WARN"]).most_common(2)
# [('INFO', 3), ('ERROR', 1)]
Path("tp/journal-capteurs.log").suffix
# '.log'
```

MODULE	POUR
`math`, `statistics`	Calcul, moyenne, médiane, écart type
`datetime`	Dates et durées : **le TP 1b**
`pathlib`, `os`	Fichiers et dossiers
`collections`	`Counter`, `defaultdict`
`csv`, `json`	Formats texte : module 2
`re`, `sqlite3`	Expressions régulières, base SQL : module 2

SÉQUENCE 6

Les principaux pièges de Python

Mutable ou immuable, et ce qu'une affectation copie vraiment.

MUTABLE OU IMMUABLE BASE

Certaines valeurs changent, d'autres sont remplacées

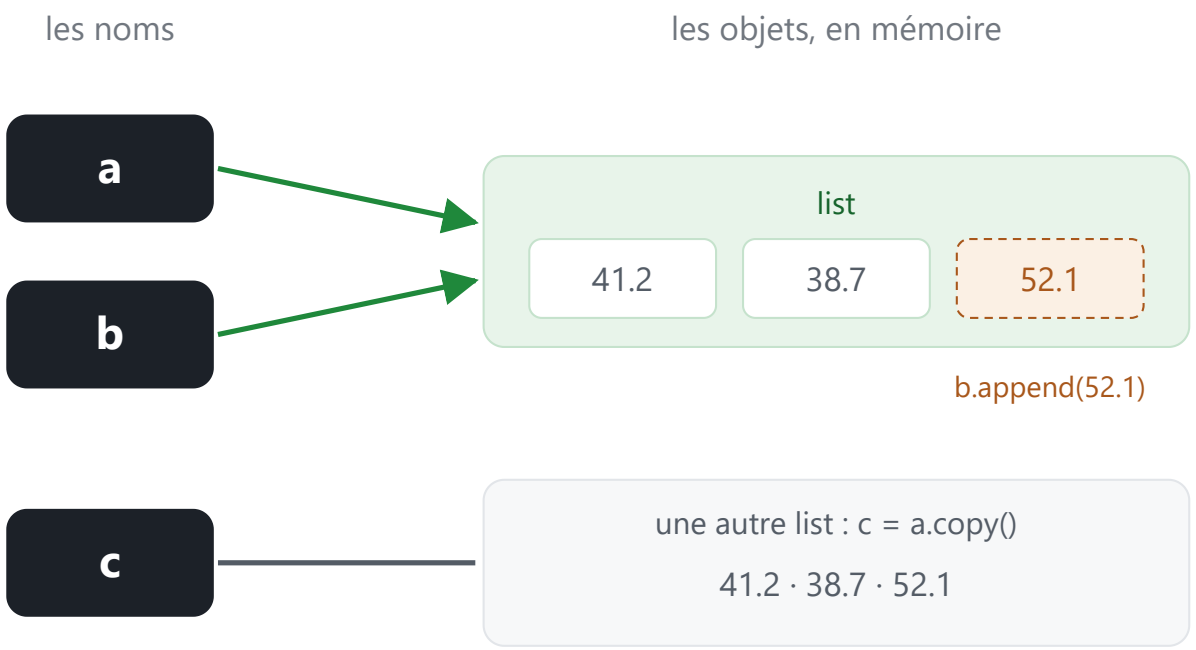
IMMUABLES	MUTABLES
`int`, `float`, `bool`	`list`
`str`	`dict`
`tuple`	`set`
`None`	la plupart des objets

```
s = "AQ"
t = s
t += "-07"
s, t
# ('AQ', 'AQ-07')
```

```
a = [41.2, 38.7]
b = a
b.append(52.1)
a
# [41.2, 38.7, 52.1]
```

> Même écriture, deux comportements. `t += "-07"` fabrique une **nouvelle** chaîne. `b.append( ... )` modifie la liste **existante**, que `a` désigne aussi.

Le signe égal colle une étiquette, il ne copie pas



Une variable est un **nom** qui désigne un objet. `b = a` ajoute un deuxième nom sur le **même** objet.

```
a is b                  # True, même objet
c = a.copy()
c.append(0)
a                       # [41.2, 38.7, 52.1]
c                       # [41.2, 38.7, 52.1, 0]
```

> Pour une copie indépendante : `a.copy()`, `list(a)` ou `a[:]`.

COPIES BASE

Copie superficielle, copie profonde

`copy()` : un seul niveau

```
grille = [[1, 2], [3, 4]]
sup = grille.copy()
sup[0][0] = 99
grille
# [[99, 2], [3, 4]]
```

La liste extérieure est neuve, mais les listes **intérieures** sont partagées.

`deepcopy()` : tous les niveaux

```
import copy
grille = [[1, 2], [3, 4]]
prof = copy.deepcopy(grille)
prof[0][0] = 99
grille
# [[1, 2], [3, 4]]
```

Tout est recopié, récursivement. Plus lent, mais indépendant.

`[[0] * 3] * 2` crée deux fois **la même** ligne : modifier l'une modifie l'autre, et l'on obtient `[[1, 0, 0], [1, 0, 0]]`. Écrire `[[0] * 3 for _ in range(2)]`.

DÉFAUT MUTABLE BASE

La liste par défaut qui se souvient

Le piège

```
def ajouter(valeur, liste=[]):
    liste.append(valeur)
    return liste

ajouter(1)      # [1]
ajouter(2)      # [1, 2]   !
```

La valeur par défaut est créée **une seule fois**, à la définition de la fonction, et partagée par tous les appels.

La correction

```
def ajouter(valeur, liste=None):
    if liste is None:
        liste = []
    liste.append(valeur)
    return liste

ajouter(1)      # [1]
ajouter(2)      # [2]
```

`None` comme sentinelle, et une liste neuve à chaque appel.

COMPARER BASE

`is` contre `==`, et les flottants

Égalité ou identité

```
a = [1, 2]
b = [1, 2]
a == b      # True, même contenu
a is b      # False, deux objets

v = None
v is None   # True, la bonne façon
```

`==` compare les **valeurs**, `is` compare les **objets**. `is` ne sert que pour `None`.

Les flottants sont approchés

```
0.1 + 0.2
# 0.30000000000000004
0.1 + 0.2 == 0.3
# False

import math
math.isclose(0.1 + 0.2, 0.3)   # True
round(0.1 + 0.2, 2)            # 0.3
```

> Ne jamais tester l'égalité de deux flottants calculés avec `==`.

Quatre surprises qu'on rencontre la première semaine

Modifier la liste qu'on parcourt

```
r = [41.2, 38.7, 52.1, 47.0]
for v in r:
    if v < 45:
        r.remove(v)
r
# [38.7, 52.1, 47.0]   38.7 a survécu !

# la bonne façon : une nouvelle liste
r = [v for v in r if v ≥ 45]
```

Les deux divisions

```
7 / 2      # 3.5
6 / 2      # 3.0    toujours un flottant
7 // 2     2
-7 // 2    3
```

L'arrondi

```
round(0.5), round(1.5), round(2.5)
# (0, 2, 2)              au pair le plus proche
round(2.675, 2)
# 2.67                   2.675 n'existe pas en binaire
```

Une chaîne ne se modifie pas

```
s = "AQ-07"
s[0] = "B"
# TypeError: 'str' object does not support
#   item assignment
"B" + s[1:]            # 'BQ-07', une nouvelle chaîne
s.replace("AQ", "BQ")  # 'BQ-07'
```

Manipuler des dates avec `datetime`

```python
from datetime import datetime, timedelta

h = datetime.strptime("2026-03-14T08:00:00",
                      "%Y-%m-%dT%H:%M:%S")
h                           # 2026-03-14 08:00:00
h.year, h.hour, h.weekday()
# (2026, 8, 5)              lundi = 0, samedi = 5
h.strftime("%d/%m/%Y %Hh")
# '14/03/2026 08h'

h + timedelta(hours=1)      # 2026-03-14 09:00:00
fin = datetime.fromisoformat("2026-03-15T07:00:00")
fin - h                     # 23:00:00
(fin - h).total_seconds() / 3600
# 23.0
```

SENS	FONCTION
Texte → date	`strptime` (*parse*), `fromisoformat`
Date → texte	`strftime` (*format*), `isoformat`
Durée	`timedelta`, ou date moins date
`%Y %m %d`	année, mois, jour
`%H %M %S`	heure, minute, seconde
`%A`	nom du jour, selon la langue du système

> Un format qui ne correspond pas lève une `ValueError` : `"2026-03-14T08:0"` ne passe pas.

Trier, compter les jours, et l'heure d'été

```python
from datetime import datetime, date, timedelta
hs = ["2026-03-14T09:00:00", "2026-03-13T23:00:00",
      "2026-03-14T08:00:00"]
d = sorted(datetime.fromisoformat(h) for h in hs)
d[0], d[-1] > d[0]
# (datetime(2026, 3, 13, 23, 0), True)

(date(2026, 3, 14) - date(2026, 1, 1)).days
# 72
[datetime(2026, 3, 14, 8) + timedelta(hours=i)
 for i in range(3)]          # 08h, 09h, 10h

datetime.strptime("14/03/26 8h05", "%d/%m/%y %Hh%M")
# 2026-03-14 08:05:00
```

```python
from zoneinfo import ZoneInfo
paris = ZoneInfo("Europe/Paris")
a = datetime(2026, 3, 29, 1, 0, tzinfo=paris)
a
# 2026-03-29 01:00:00+01:00
a + timedelta(hours=2)
# 2026-03-29 03:00:00+02:00   heure murale
utc = ZoneInfo("UTC")
(a.astimezone(utc) + timedelta(hours=2)).astimezone(paris)
# 2026-03-29 04:00:00+02:00   2 h réellement écoulées

datetime(2026, 3, 14, 8) < a
# TypeError: can't compare offset-naive and
#   offset-aware datetimes
```

`%y`	année sur deux chiffres	`%j`	jour de l'année
`%b`	mois abrégé	`%W`	numéro de semaine
`%a`	jour abrégé	`%z`	décalage horaire

> Une date **naïve** n'a pas de fuseau, une date **consciente** en a un. On ne mélange pas les deux. Règle simple : stocker et calculer en **UTC**, convertir pour l'affichage.

TRAVAUX PRATIQUES 1B

Un module à soi, et les dates

≈ 50 min · Individuel · Restitution 5 min · `tp/tp1b-modules-dates/`

1. **Un module à soi**

   Ranger les fonctions du TP 1a dans `outils.py`, les tester sous `if __name__ == "__main__"`.

2. **L'importer**

   `from outils import plus_longue_serie` dans un autre script : le bloc de test ne s'exécute pas.

3. **Lire les horodatages**

   `horodatages-aq03.txt`, une ligne par mesure reçue : `datetime.strptime`.

4. **Compter et chercher**

   Les mesures par jour de la semaine, puis les heures manquantes du capteur.

Les fichiers

`tp/tp1b-modules-dates/` :
`enonce.md` · `depart.py` · `horodatages-aq03.txt`
`corrige.py`, projeté en restitution

Un résultat pour se vérifier

162 horodatages, du 02/03 00:00 au 08/03 23:00 ; **6** heures manquantes, dont 3 le mardi 3 mars.

Bonus

Combien de temps sépare la première et la dernière mesure ? Un `timedelta` : `6 days, 23:00:00`.

CONTRÔLE DES ACQUIS

Cinq questions avant de fermer le module 1

Q1 Vous lisez `"41.2"` dans un fichier et faites `valeur * 2`. Qu'obtenez-vous ?

`'41.241.2'` : une chaîne multipliée par un entier est répétée. Il fallait d'abord `float(valeur)`.

Q2 Pourquoi `releves = releves.sort()` vide-t-il vos données ?

`sort()` trie la liste sur place et renvoie `None`, que l'on affecte à `releves`. Utiliser `releves.sort()` seul, ou `sorted(releves)`.

Q3 `b = a`, puis `b.append(0)`. `a` a-t-il changé ?

Oui, si `a` est une liste : `a` et `b` sont deux noms pour le même objet. Pour une copie, `a.copy()`.

Q4 Quel est le problème de `def f(x, cache={})` ?

Le dictionnaire par défaut est créé une seule fois et partagé entre tous les appels. Mettre `None` par défaut et créer le dictionnaire dans la fonction.

Q5 À quoi sert `if __name__ == "__main__":` ?

À n'exécuter un bloc que lorsque le fichier est lancé comme script, pas quand il est importé comme module.

CONTRÔLE DES ACQUIS

Le quiz en ligne du module 1

Il porte sur ce que le module vient de couvrir : types, structures de contrôle, fonctions, modules, mutable et immuable.

```
wytasoft.com/quizz/python-traitement-donnees-module-1-fr
```

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

ANNEXES DU MODULE 1 AVANCÉ

Un code que les autres relisent

Le guide de style de Python, et les outils qui l'appliquent pour vous.

STYLE AVANCÉ

La PEP 8 et les outils qui l'appliquent

ÉLÉMENT	CONVENTION
Variables, fonctions	`moyenne_glissante`
Classes	`Capteur` (module 2)
Constantes	`SEUIL_NO2`
Indentation	4 espaces
Commentaire en fin de ligne	deux espaces, `#`, un espace

```
# ruff : vérifier, puis mettre en forme
ruff check analyse.py
ruff format analyse.py
```

Laisser faire les outils

ruff ou **black** mettent en forme automatiquement ; **ruff** et **pylint** signalent les erreurs probables, dont le défaut mutable.

Commenter le pourquoi

Le code dit **ce qu'il fait** ; le commentaire dit **pourquoi**. `# seuil OMS 2021 du NO2 sur 24 h : 25 µg/m³` vaut mieux que `# compare à 25`.

MODULE 2

Compléments sur le langage

Les classes, les fichiers texte, une base SQLite et les expressions régulières, pour transformer le journal en CSV.

WYTASOFT
Unlock the power of your data

JOUR 2 · MODULE 2

Compléments sur le langage

Des objets, des fichiers, une base de données et des expressions régulières : de quoi transformer un journal brut en un CSV propre.

DURÉE

≈ 1 h 55 d'exposé + 2 TP (1 h 15)

À LA FIN

Écrire une classe, lire et écrire des fichiers, interroger SQLite, extraire avec une regex

TP

2a : une classe et un fichier · 2b : du journal à `mesures.csv`

BIENVENUE AU JOUR 2

Ce qu'on garde d'hier

Des types, et leurs méthodes

`str`, `list`, `dict` savent faire des choses : `"aq-07".upper()`, `mesures.append(41.2)`. Ce matin, on comprend d'où viennent ces méthodes.

Mutable ou immuable

Une liste se modifie en place, une chaîne jamais. Deux noms peuvent désigner **la même** liste : le piège revient ce matin, dans les classes.

Des fonctions, des modules

`def`, paramètres par défaut, `import`. Une classe, c'est la suite logique : des fonctions rangées avec les données qu'elles manipulent.

Hier, vous avez appris à **écrire** du Python. Aujourd'hui, vous apprenez à le **brancher** sur le monde : des fichiers, une base, du texte à déchiffrer.

FEUILLE DE ROUTE

Ce que couvre le module 2

1 **La syntaxe orientée objet**
Tout est objet. Créer une classe : attributs, méthodes, méthodes spéciales.

2 **Lire et écrire des fichiers texte**
`open`, `with`, l'encodage, le module `csv`.

TP **TP 2a**
Une classe `Capteur` et un extrait du journal, lu avec `split()`.

3 **La bibliothèque standard**
`sqlite3` pour la base relationnelle, `re` pour les expressions régulières.

TP **TP 2b**
Du journal brut des capteurs à `mesures.csv`.

Le fil de la matinée

Air'Cité a deux sources : une base `capteurs.db` qui dit **où** est chaque capteur, et un journal `journal-capteurs.log` qui dit **ce qu'il mesure**.
La question : **comment en faire une seule table, propre, sans rien perdre en silence ?**

SÉQUENCE 1

La syntaxe orientée objet

Des données et le code qui les manipule, rangés ensemble.

TOUT EST OBJET BASE

Chaque valeur a un type, et le type apporte des méthodes

```
type(42), type("AQ-07"), type([1, 2]), type(len)
# (<class 'int'>, <class 'str'>, <class 'list'>,
#  <class 'builtin_function_or_method'>)

"aq-07".upper()          # 'AQ-07'
(3.5).is_integer()       # False

[m for m in dir("aq") if not m.startswith("_")][:8]
# ['capitalize', 'casefold', 'center', 'count',
#  'encode', 'endswith', 'expandtabs', 'find']

isinstance(True, int)    # True
```

Un **objet**, c'est une valeur *plus* les opérations qu'elle sait faire.

Une **classe**, c'est le moule : `str` est la classe, `"AQ-07"` en est une **instance**.

Même une fonction est un objet : on peut la ranger dans une liste, la passer en argument.

> `dir(x)` et `help(x)` : les deux réflexes pour découvrir ce qu'un objet sait faire, sans quitter Python.

POURQUOI DES CLASSES BASE

Un dictionnaire range des données, une classe range aussi le code

Un dictionnaire et des fonctions à part

```
capteur = {"id": "AQ-07", "quartier": "Port",
           "mesures": [41.2, 44.9]}

def moyenne(c):
    return sum(c["mesures"]) / len(c["mesures"])

moyenne(capteur)
capteur["mesure"].append(50)   # KeyError
```

Rien ne dit quelles clés existent. Une faute de frappe se découvre à l'exécution, loin de sa cause.

Une classe

```
class Capteur:
    def __init__(self, id, quartier):
        self.id = id
        self.quartier = quartier
        self.mesures = []

    def moyenne(self):
        return sum(self.mesures) / len(self.mesures)
```

La forme d'un capteur est écrite **une fois**, et le calcul est rangé avec les données qu'il utilise.

> La règle pratique : dès que plusieurs fonctions passent leur temps à se repasser le même dictionnaire, c'est une classe qui attend d'être écrite.

CRÉER UNE CLASSE BASE

`class`, `__init__` et `self`

```python
class Capteur:
    def __init__(self, id, quartier):
        self.id = id
        self.quartier = quartier
        self.mesures = []

c = Capteur("AQ-07", "Port")
c.id, c.quartier, c.mesures
# ('AQ-07', 'Port', [])

type(c)
# <class '__main__.Capteur'>
print(c)
# <__main__.Capteur object at 0x000002184AC37210>
```

1. **`Capteur("AQ-07", "Port")`**

   Appeler la classe crée une instance vide…

2. **`__init__(self, …)`**

   … puis Python appelle `__init__` en passant cette instance comme premier argument.

3. **`self.id = id`**

   On accroche les données à l'instance : ce sont ses **attributs d'instance**.

> `self` n'est pas un mot-clé, c'est une convention. Mais l'oublier dans `def moyenne(self)` donne une `TypeError` à l'appel.

ATTRIBUTS BASE

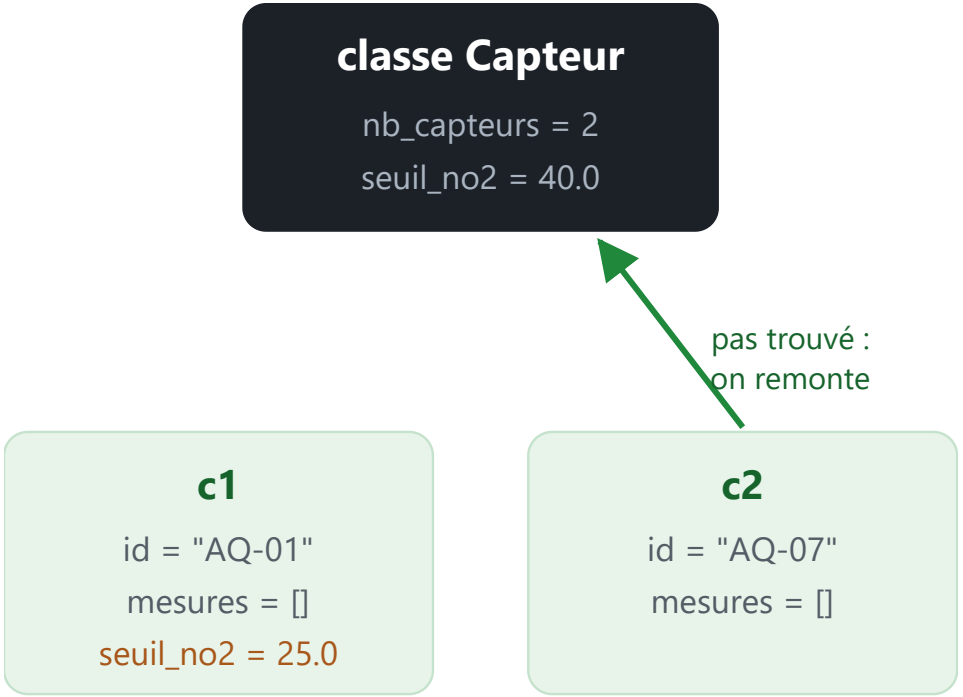
Attribut de classe, attribut d'instance

```python
class Capteur:
    nb_capteurs = 0          # attributs de classe :
    seuil_no2 = 40.0         # partagés par toutes les instances

    def __init__(self, id, quartier):
        self.id = id                    # attributs d'instance :
        self.quartier = quartier        # propres à chacune
        self.mesures = []
        Capteur.nb_capteurs += 1

c1 = Capteur("AQ-01", "Centre")
c2 = Capteur("AQ-07", "Port")
Capteur.nb_capteurs, c1.seuil_no2      # (2, 40.0)

c1.seuil_no2 = 25.0
c1.seuil_no2, c2.seuil_no2, Capteur.seuil_no2
# (25.0, 40.0, 40.0)
```



Python cherche d'abord dans l'**instance**, puis dans la **classe**. Affecter `c1.seuil_no2` crée un attribut d'instance qui **masque** celui de la classe.

LE PIÈGE BASE

La liste rangée au mauvais étage

Attribut de classe : une liste pour tous

```
class CapteurFaux:
    mesures = []

    def __init__(self, id):
        self.id = id

    def ajouter(self, v):
        self.mesures.append(v)

a = CapteurFaux("AQ-01")
b = CapteurFaux("AQ-02")
a.ajouter(41.2)
b.mesures                  # [41.2]
a.mesures is b.mesures     # True
```

Attribut d'instance : une liste chacun

```
class Capteur:
    def __init__(self, id):
        self.id = id
        self.mesures = []   # créée à chaque instance

    def ajouter(self, v):
        self.mesures.append(v)
```

`append` **modifie** la liste trouvée : il ne crée pas d'attribut d'instance. À gauche, il n'existe qu'une liste, celle de la classe.

C'est le cousin du piège d'hier, `def f(x, acc=[])` : un objet mutable créé **une seule fois** et partagé ensuite. Les listes et dictionnaires se créent dans `__init__`.

MÉTHODES BASE

Des méthodes, et une propriété

```python
class Capteur:
    seuil_no2 = 40.0

    def __init__(self, id, quartier):
        self.id = id
        self.quartier = quartier
        self.mesures = []

    def ajouter(self, no2):
        self.mesures.append(no2)

    def moyenne(self):
        if not self.mesures:
            return None
        return sum(self.mesures) / len(self.mesures)

    @property
    def en_alerte(self):
        m = self.moyenne()
        return m is not None and m > self.seuil_no2
```

```python
c = Capteur("AQ-07", "Port")
c.ajouter(41.2)
c.ajouter(44.9)
round(c.moyenne(), 2)    # 43.05
c.en_alerte              # True, sans parenthèses

c.en_alerte = True
# AttributeError: property 'en_alerte'
# of 'Capteur' object has no setter
```

Une **méthode** est une fonction définie dans la classe : elle reçoit l'instance en `self`.

`@property` fait d'une méthode un **attribut calculé** : il se lit comme une donnée, se recalcule à chaque lecture, et ne s'écrit pas.

MÉTHODES DE CLASSE BASE

`@classmethod`, `@staticmethod`, et le `_` qui dit « interne »

```python
class Capteur:
    def __init__(self, id, quartier):
        self.id = id
        self.quartier = quartier
        self._mesures = []          # interne, par convention

    @classmethod
    def depuis_ligne(cls, ligne, quartier_de):
        id = ligne.split()[2]
        return cls(id, quartier_de[id])

    @staticmethod
    def id_valide(id):
        return re.fullmatch(r"AQ-\d{2}", id) is not None
```

```python
ligne = "2026-03-14T08:00:00 INFO  AQ-07 no2=41.2 …"
Capteur.depuis_ligne(ligne, {"AQ-07": "Port"})
# Capteur('AQ-07', 'Port')
Capteur.id_valide("AQ-07"), Capteur.id_valide("AQ-7")
# (True, False)
```

DÉCORATEUR	REÇOIT	SERT À
aucun	`self`, l'instance	travailler sur ses données
`@classmethod`	`cls`, la classe	un **autre constructeur**
`@staticmethod`	rien	une fonction rangée dans la classe

> Python n'a pas de `private`. Un nom qui commence par `_` signifie « usage interne, ne pas toucher » : rien ne l'interdit, c'est un contrat entre développeurs.

Vous en utilisez déjà : `dict.fromkeys(…)` et `datetime.fromisoformat(…)` sont des méthodes de classe.

MÉTHODES SPÉCIALES BASE

`__repr__` et `__str__` : un objet qui se présente

```
class Capteur:
    ...
    def __repr__(self):
        return f"Capteur({self.id!r}, {self.quartier!r})"

    def __str__(self):
        return f"{self.id} ({self.quartier}) : {len(self.mesures)} mesures"
```

```
c                  # dans un notebook ou l'interpréteur
# Capteur('AQ-07', 'Port')
print(c)
# AQ-07 (Port) : 2 mesures
[c]                # une liste affiche le repr
# [Capteur('AQ-07', 'Port')]
```

	POUR QUI	APPELÉE PAR
`__repr__`	Le développeur : sans ambiguïté, idéalement du code valide	`repr()`, l'interpréteur, les conteneurs
`__str__`	L'utilisateur : lisible	`print()`, `str()`, les f-strings

> Les méthodes entourées de `__` (« dunder ») ne s'appellent pas directement : Python les appelle pour vous quand vous utilisez `print`, `len`, `==`, `<` …

Sans `__str__`, `print` se rabat sur `__repr__`. Si vous n'en écrivez qu'une, écrivez `__repr__`.

MÉTHODES SPÉCIALES BASE

Comparer, trier, mesurer

```python
class Capteur:
    ...
    def __len__(self):
        return len(self.mesures)

    def __eq__(self, autre):
        return isinstance(autre, Capteur) and self.id == autre.id

    def __lt__(self, autre):
        return self.id < autre.id
```

```python
len(c)                                  # 2
c == Capteur("AQ-07", "Port")           # True
c is Capteur("AQ-07", "Port")           # False

reseau = [Capteur("AQ-10", "Université"), c,
          Capteur("AQ-01", "Centre")]
sorted(reseau)
# [Capteur('AQ-01', 'Centre'), Capteur('AQ-07', 'Port'),
#  Capteur('AQ-10', 'Université')]
```

VOUS ÉCRIVEZ	PYTHON APPELLE
`len(c)`	`c.__len__()`
`c == d`	`c.__eq__(d)`
`c < d`, `sorted`, `max`	`c.__lt__(d)`
`print(c)`	`c.__str__()`

> `==` compare les **valeurs**, selon votre `__eq__`. `is` compare les **identités** : est-ce le même objet en mémoire ?

Sans `__eq__`, `==` se comporte comme `is` : deux capteurs AQ-07 créés séparément seraient différents.

HÉRITAGE SI LE TEMPS LE PERMET

Une classe qui en prolonge une autre

```python
class CapteurMeteo(Capteur):
    def __init__(self, id, quartier, altitude):
        super().__init__(id, quartier)
        self.altitude = altitude

    def __repr__(self):
        return f"CapteurMeteo({self.id!r}, alt={self.altitude})"

m = CapteurMeteo("AQ-12", "Université", 212)
m.ajouter(18.0)          # méthode héritée
m.moyenne()              # 18.0
m                        # CapteurMeteo('AQ-12', alt=212)
print(m)                 # AQ-12 (Université) : 1 mesures
isinstance(m, Capteur)   # True
```

`CapteurMeteo(Capteur)` : la sous-classe **hérite** de tout ce que sait faire `Capteur`.

Elle peut **ajouter** (`altitude`) et **redéfinir** (`__repr__`). Ce qu'elle ne redéfinit pas, ici `__str__`, vient du parent.

`super().__init__(…)` appelle l'initialisation du parent : sans elle, pas de `self.mesures`.

> En traitement de données, on hérite rarement soi-même. On le croise dans les bibliothèques : en Python, toutes les exceptions héritent d'`Exception`.

Le typage canard : ce qui compte, c'est ce que l'objet sait faire

```python
class JournalFichier:
    def __init__(self, chemin):
        self.chemin = chemin
    def lignes(self):
        with open(self.chemin, encoding="utf-8") as f:
            for ligne in f:
                yield ligne.rstrip("\n")

class JournalMemoire:
    def __init__(self, lignes):
        self._lignes = lignes
    def lignes(self):
        return iter(self._lignes)

def compter_erreurs(journal):
    return sum(" ERROR " in l for l in journal.lignes())

compter_erreurs(JournalFichier("journal-capteurs.log"))   # 1
compter_erreurs(JournalMemoire(["… ERROR AQ-03 timeout"]))  # 1
compter_erreurs("journal-capteurs.log")
# AttributeError: 'str' object has no attribute 'lignes'
```

Les deux classes n'ont **aucun parent commun**. Elles ont la même méthode `lignes()`, et cela suffit à `compter_erreurs`.

« Si ça marche comme un canard et que ça cancane comme un canard, c'est un canard » : Python ne vérifie pas le type, il appelle la méthode.

> C'est ce qui fait marcher `len()` sur une chaîne, une liste ou votre `Capteur`, et `for` sur une liste, un fichier ou un générateur.

SÉQUENCE 2

Lire et écrire des fichiers texte

Ouvrir, lire ligne à ligne, écrire, et ne jamais oublier l'encodage.

OUVRIR UN FICHIER BASE

`open`, `with`, et les trois modes

```
with open("rapport.txt", "w", encoding="utf-8") as f:
    f.write("Capteurs en alerte\n")
    f.write("AQ-04 Parvis de la Gare\n")

with open("rapport.txt", "a", encoding="utf-8") as f:
    print("AQ-01 Hôtel de Ville", file=f)

with open("rapport.txt", encoding="utf-8") as f:
    contenu = f.read()
print(contenu)
# Capteurs en alerte
# AQ-04 Parvis de la Gare
# AQ-01 Hôtel de Ville

f.closed     # True : with a fermé le fichier
```

MODE	EFFET
`"r"`	Lire. Par défaut. Erreur si le fichier n'existe pas
`"w"`	Écrire. **Vide** le fichier s'il existe
`"a"`	Ajouter à la fin

> `with` ferme le fichier en sortie de bloc, **même si une erreur survient**. Un fichier jamais fermé peut perdre ses dernières lignes écrites.

`write` n'ajoute pas de retour à la ligne : `\n` est à votre charge. `writelines(liste)` non plus. `print(…, file=f)` l'ajoute.

LE PIÈGE WINDOWS BASE

Toujours préciser `encoding="utf-8"`

Sans `encoding`, Python utilise l'encodage **du système**. Sous Windows en français, c'est `cp1252`, pas UTF-8 :

```
import locale
locale.getpreferredencoding(False)   # 'cp1252'

# un fichier UTF-8 lu sans encoding
open("capteurs.txt").read()
# 'HÃ´tel de Ville\n'

# un fichier écrit sans encoding, relu en UTF-8
open("capteurs.txt", encoding="utf-8").read()
# UnicodeDecodeError: 'utf-8' codec can't decode
# byte 0xc9 in position 0: invalid continuation byte
```

Pourquoi c'est sournois

Le script marche sur le Mac du collègue, et sur Windows tant que les données n'ont pas d'accent. Le premier « Écluse Nord » le casse, ou pire : le transforme en « Ã‰cluse » sans erreur.

La règle

`encoding="utf-8"` à chaque `open` en mode texte. Pour un CSV ouvert ensuite dans Excel, `"utf-8-sig"` ajoute la marque qu'Excel attend.

LIRE BASE

Ligne à ligne : la mémoire reste constante

```
with open("journal-capteurs.log", encoding="utf-8") as f:
    for numero, ligne in enumerate(f, start=1):
        ligne = ligne.rstrip("\n")
        if "ERROR" in ligne:
            print(numero, ligne)
# 3 2026-03-14T08:00:00 ERROR AQ-03 timeout
```

MÉTHODE	REND	MÉMOIRE
`for ligne in f`	une ligne à la fois	une ligne
`f.read()`	tout le fichier, une chaîne	tout le fichier
`f.readlines()`	une liste de lignes	tout le fichier

Un fichier ouvert est **itérable** : une boucle `for` le lit ligne par ligne, sans le charger en entier.

Chaque ligne garde son `\n` final : `rstrip("\n")` l'enlève, `strip()` enlève aussi les espaces.

`enumerate(f, start=1)` donne le **numéro de ligne** : indispensable pour signaler une ligne rejetée.

> Un journal de 5 Go se lit ainsi sur un portable de 8 Go. Avec `read()`, non.

`pathlib` : des chemins qui marchent partout

```
from pathlib import Path

journal = Path("journal-capteurs.log")
journal.exists(), journal.suffix, journal.stem
# (True, '.log', 'journal-capteurs')

dossier = Path("donnees")
dossier.mkdir(exist_ok=True)
cible = dossier / "mesures.csv"
print(cible)              # donnees\mesures.csv  (sous Windows)
cible.as_posix()          # 'donnees/mesures.csv'

lignes = journal.read_text(encoding="utf-8").splitlines()
sorted(p.name for p in Path(".").glob("*.log"))
# ['journal-capteurs.log']
```

L'opérateur `/` assemble les morceaux avec le bon séparateur : `\` sous Windows, `/` ailleurs.

`glob("*.log")` liste les fichiers d'un dossier, `read_text` et `write_text` lisent ou écrivent d'un coup.

> Sous Windows, `"C:\new\tp"` contient un retour à la ligne (`\n`) et une tabulation (`\t`). Écrire `Path(r"C:\new\tp")`, ou des `/`.

Écrire et relire un CSV sans le bricoler

```python
import csv

colonnes = ["horodatage", "capteur", "quartier",
            "no2", "pm10", "temp"]
with open("mesures.csv", "w", encoding="utf-8",
          newline="") as f:
    w = csv.DictWriter(f, fieldnames=colonnes)
    w.writeheader()
    w.writerows(lignes)    # une liste de dict

with open("mesures.csv", encoding="utf-8",
          newline="") as f:
    for ligne in csv.DictReader(f):
        print(ligne["capteur"], float(ligne["no2"]))
# AQ-07 41.2
# AQ-01 52.8
```

Sans `newline=""`, sous Windows

```
b'a,b\r\r\n1,2\r\r\n'
[['a', 'b'], [], ['1', '2'], []]
```

Le module écrit déjà `\r\n` ; le mode texte de Windows en ajoute un `\r`. Résultat : une ligne vide sur deux.

Pourquoi pas `",".join(…)`

Un nom contenant une virgule casse tout. `csv` met les guillemets qu'il faut :

```
AQ-02,"Place des Halles, Centre"
```

Tout ce que lit `csv` est une **chaîne** : `float(…)` est à votre charge.

JSON : des dictionnaires et des listes, écrits en texte

```python
import json

seuils = {"seuil_no2": 40.0, "capteur": "Écluse Nord",
          "actif": True, "note": None}
with open("seuils.json", "w", encoding="utf-8") as f:
    json.dump(seuils, f, ensure_ascii=False, indent=2)

with open("seuils.json", encoding="utf-8") as f:
    relu = json.load(f)
relu == seuils                          # True
```

```json
{
  "seuil_no2": 40.0,
  "capteur": "Écluse Nord",
  "actif": true,
  "note": null
}
```

PYTHON	JSON	RELU EN
`dict`	objet `{}`	`dict`, clés **toujours** en `str`
`list`, `tuple`	tableau `[]`	`list`
`True`, `None`	`true`, `null`	`True`, `None`
`datetime`	refusé	`TypeError`

```python
print(json.dumps({"capteur": "Écluse"}))  # {"capteur": "\u00c9cluse"}
print(json.dumps({1: "a"}))               # {"1": "a"}
json.loads(json.dumps(("Centre", "Port")))  # ['Centre', 'Port']
```

`dump` / `load` : un fichier. `dumps` / `loads` (s comme *string*) : une chaîne.

TRAVAUX PRATIQUES 2A

Une classe et un fichier

≈ 25 min · Individuel · Restitution 5 min · `tp/tp2a-classes-fichiers/`

1. **La classe `Capteur`**

   Une liste de mesures **par instance**, `ajouter` qui convertit ou compte un manque, `moyenne` qui rend `None` à vide, un `__repr__`.

2. **Lire l'extrait du journal**

   `with open(…, encoding="utf-8")`, ligne à ligne, `split()` : on garde les lignes `INFO` de 6 champs, on **compte** les autres.

3. **Écrire `resume.csv`**

   Une ligne par capteur, triée : mesures, manquantes, NO_2 moyen et maximum, avec `csv.writer` et `newline=""`.

Les fichiers

`enonce.md` · `depart.py` (vérifications OK / ÉCHEC)
`extrait-journal.log` : les 120 premières lignes du journal

Pour se vérifier

12 capteurs, 4 lignes écartées ; `Capteur('AQ-01', 9 mesures)` et une valeur manquante ; moyenne d'AQ-03 : 34,1.

SÉQUENCE 3

La bibliothèque standard : `sqlite3` et `re`

Une base relationnelle et des expressions régulières, sans rien installer.

SQLITE3 BASE

Se connecter, exécuter, récupérer

```python
import sqlite3

conn = sqlite3.connect("capteurs.db")
cur = conn.cursor()
cur.execute("SELECT id, nom, quartier FROM capteurs "
            "WHERE quartier = 'Port'")
cur.fetchall()
# [('AQ-07', 'Quai des Docks', 'Port'),
#  ('AQ-08', 'Écluse Nord', 'Port'),
#  ('AQ-09', 'Bassin Sud', 'Port')]

cur.execute("SELECT quartier, COUNT(*) FROM capteurs "
            "GROUP BY quartier ORDER BY quartier")
for quartier, n in cur:
    print(quartier, n)
# Centre 3
# Gare 3
# Port 3
# Université 3

conn.execute("SELECT COUNT(*) FROM capteurs").fetchone()  # (12,)
conn.close()
```

1. **`connect`**

   Ouvre le fichier de la base. Le crée s'il n'existe pas : une faute de frappe dans le nom donne une base vide, sans erreur.

2. **`execute`**

   Envoie une requête SQL. Le curseur garde le résultat.

3. **`fetchall` / `fetchone` / boucle**

   Chaque ligne revient comme un **tuple**.

> SQLite est une base complète dans **un seul fichier**, sans serveur. Pour PostgreSQL ou Oracle, le pilote change, la démarche reste : connexion, curseur, requête, résultats (norme DB-API, PEP 249).

SQLITE3 BASE

Les paramètres `?` : jamais de f-string dans le SQL

La valeur collée dans la requête

```
saisie = "x' OR '1'='1"
sql = ("SELECT id FROM capteurs "
       f"WHERE quartier = '{saisie}'")
len(conn.execute(sql).fetchall())        # 12
```

La saisie a **réécrit** la requête : la condition devient toujours vraie, les 12 capteurs sortent. C'est une **injection SQL**.

La valeur passée en paramètre

```
sql = "SELECT id FROM capteurs WHERE quartier = ?"
len(conn.execute(sql, (saisie,)).fetchall())   # 0

conn.execute(sql, ("Gare",)).fetchall()
# [('AQ-04',), ('AQ-05',), ('AQ-06',)]
```

Le pilote transmet la valeur **à part** : elle ne peut pas changer la structure de la requête. Les guillemets et accents sont gérés.

> Le second argument est un **tuple**, même pour une seule valeur : `("Gare",)`, avec la virgule. `("Gare")` est une simple chaîne de quatre caractères, et l'appel échoue.

Des lignes nommées, et des écritures validées

```
conn = sqlite3.connect("capteurs.db")
conn.row_factory = sqlite3.Row
r = conn.execute("SELECT * FROM capteurs WHERE id = ?",
                 ("AQ-07",)).fetchone()
r["nom"], r["lat"]       # ('Quai des Docks', 45.748)
dict(r)
# {'id': 'AQ-07', 'nom': 'Quai des Docks',
#  'quartier': 'Port', 'lat': 45.748, 'lon': 4.827,
#  'mise_en_service': '2024-09-02'}

quartier_de = dict(conn.execute(
    "SELECT id, quartier FROM capteurs"))
quartier_de["AQ-10"]     # 'Université'
```

```
with conn:   # commit, ou rollback si erreur
    conn.execute("CREATE TABLE IF NOT EXISTS alertes"
                 "(capteur TEXT, horodatage TEXT, no2 REAL)")
    cur = conn.executemany(
        "INSERT INTO alertes VALUES (?, ?, ?)",
        [("AQ-04", "2026-03-14T08:00:00", 61.0),
         ("AQ-01", "2026-03-14T08:00:00", 52.8)])
cur.rowcount   # 2 lignes insérées
conn.close()   # with ne ferme pas la connexion
```

> Sans `commit`, une écriture est **perdue** à la fermeture. `with conn:` valide en sortie de bloc, ou annule tout si une erreur survient. Mais il ne ferme pas la connexion : `close()` reste à faire.

Le dictionnaire `quartier_de` est la jointure du TP : une requête, puis une recherche instantanée par identifiant.

AU-DELÀ DE SQLITE · SI LE TEMPS LE PERMET

Les autres bases : le pilote change, le code reste

BASE	PILOTE À INSTALLER	PARAMÈTRE
SQLite	`sqlite3`, livré avec Python	`?`
PostgreSQL	`psycopg`	`%s`
Oracle	`oracledb`	`:nom`
SQL Server	`pyodbc`	`?`
MySQL	`mysql-connector-python`	`%s`

Tous suivent la **DB-API 2.0** (PEP 249) : `connect`, `cursor`, `execute`, `fetchall`, `commit`, `close`. Seuls la chaîne de connexion et le style de paramètre changent.

Le pont vers le module 4

```
import pandas as pd
conn = sqlite3.connect("capteurs.db")
pd.read_sql("SELECT id, quartier, lat FROM capteurs "
            "WHERE quartier = ?", conn, params=("Port",))
#       id quartier      lat
# 0  AQ-07     Port  45.7480
# 1  AQ-08     Port  45.7502
# 2  AQ-09     Port  45.7465
```

SQLAlchemy ajoute une couche commune à toutes ces bases : une seule chaîne de connexion (`"postgresql+psycopg://…"`), et c'est ce que pandas attend pour toute base autre que SQLite.

EXPRESSIONS RÉGULIÈRES BASE

`re` : chercher un motif plutôt qu'un texte

```
import re
ligne = "2026-03-14T08:00:00 INFO  AQ-07 no2=41.2 pm10=23.5 temp=11.8"

re.match(r"AQ-\d\d", ligne)     # None : pas au début
re.search(r"AQ-\d\d", ligne)
# <re.Match object; span=(26, 31), match='AQ-07'>
re.search(r"AQ-\d\d", ligne).group()     # 'AQ-07'

re.findall(r"\d+\.\d+", ligne)
# ['41.2', '23.5', '11.8']
dict(re.findall(r"(\w+)=([\d.]+)", ligne))
# {'no2': '41.2', 'pm10': '23.5', 'temp': '11.8'}
```

FONCTION	CHERCHE	REND
`re.match`	au début seulement	un `Match` ou `None`
`re.search`	n'importe où	le premier `Match` ou `None`
`re.findall`	partout	une liste

MOTIF	SIGNIFIE
`\d`	un chiffre
`\w`	lettre, chiffre ou `_`
`\s`	un blanc
`.` / `\.`	n'importe quoi / un point
`+ * ?`	1 ou plus, 0 ou plus, 0 ou 1
`{2}`	exactement 2 fois
`[\d.]`	un chiffre ou un point
`^ $`	début, fin
`( )`	un groupe à extraire

Groupes nommés et motifs compilés

```
m = re.search(r"(?P<capteur>AQ-\d{2}) no2=(?P<no2>[\d.]+)",
              ligne)
m.group("capteur"), m.group("no2")   # ('AQ-07', '41.2')
m.groupdict()
# {'capteur': 'AQ-07', 'no2': '41.2'}
```

```
MESURE = re.compile(
    r"^(\S+) INFO\s+(AQ-\d{2}) "
    r"no2=([\d.]+) pm10=([\d.]+) temp=(-?[\d.]+)$")

MESURE.match(ligne).groups()
# ('2026-03-14T08:00:00', 'AQ-07', '41.2', '23.5', '11.8')
```

`(?P<nom>…)` donne un nom au groupe : on lit `m["no2"]` au lieu de `m.group(3)`, et `groupdict()` rend directement un dictionnaire.

`re.compile` prépare le motif une fois : on le nomme, on le réutilise dans la boucle.

> `^` et `$` obligent la regex à décrire **toute** la ligne. Une ligne tronquée ne correspond plus : elle est rejetée au lieu d'être lue à moitié.

`re.VERBOSE` : la regex du journal Air'Cité

```
MESURE = re.compile(r"""
    ^(?P<horodatage>\d{4}-\d\d-\d\dT\d\d:\d\d:\d\d)
    \s+INFO\s+
    (?P<capteur>AQ-\d{2})
    \s+no2=(?P<no2>[\d.]*)        # vide si la mesure manque
    \s+pm10=(?P<pm10>[\d.]*)
    \s+temp=(?P<temp>-?[\d.]*)$
""", re.VERBOSE)

for ligne in open("journal-capteurs.log", encoding="utf-8"):
    m = MESURE.match(ligne.rstrip("\n"))
    print("OK   " if m else "REJET", ligne.rstrip()[20:])
```

```
OK    INFO  AQ-07 no2=41.2 pm10=23.5 temp=11.8
OK    INFO  AQ-01 no2=52.8 pm10=30.1 temp=12.4
REJET ERROR AQ-03 timeout
OK    INFO  AQ-10 no2= pm10=18.2 temp=11.1
REJET INFO  AQ-04 no2=61.0 pm10=34
OK    INFO  AQ-07 no2=44.9 pm10=25.0 temp=12.6
```

En mode `VERBOSE`, les blancs du motif sont ignorés et `#` ouvre un commentaire : la regex se lit sur plusieurs lignes.

Un espace réel s'écrit alors `\s` ou `\ `.

Trois cas, trois décisions

Erreur du capteur : rejetée, comptée à part.
Valeur manquante (`no2=`) : gardée, cellule vide.
Ligne tronquée : rejetée.

Remplacer, découper, et le piège du `.*` gourmand

Remplacer avec `re.sub`

```
re.sub(r"\s+", " ", ligne)
# '2026-03-14T08:00:00 INFO AQ-07 no2=41.2 …'
re.sub(r"AQ-(\d{2})", r"capteur \1", "AQ-07 et AQ-10")
# 'capteur 07 et capteur 10'
```

Découper avec `re.split`

```
"Centre; Gare,Port ,  Université".split(",")
# ['Centre; Gare', 'Port ', '  Université']
re.split(r"\s*[;,]\s*", "Centre; Gare,Port ,  Université")
# ['Centre', 'Gare', 'Port', 'Université']
```

Gourmand ou paresseux

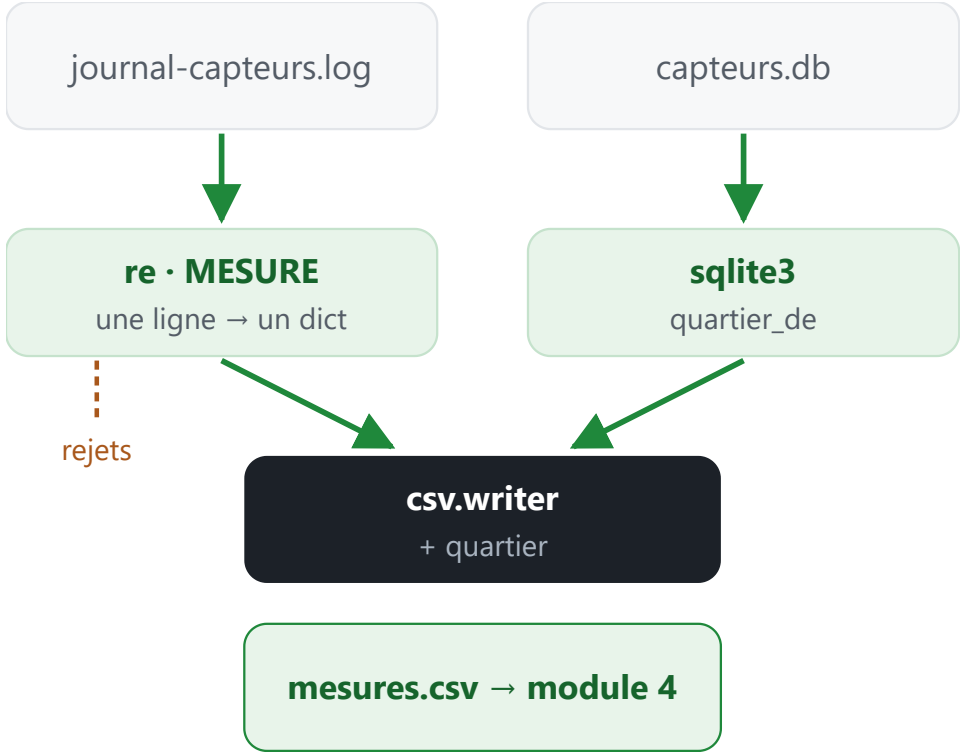
```
html = "<b>AQ-07</b> et <b>AQ-10</b>"
re.findall(r"<b>(.*)</b>", html)
# ['AQ-07</b> et <b>AQ-10']
re.findall(r"<b>(.*?)</b>", html)
# ['AQ-07', 'AQ-10']
```

Les options

```
re.findall(r"error", "ERROR Error error",
           flags=re.IGNORECASE)
# ['ERROR', 'Error', 'error']
re.findall(r"\bAQ-\d\d\b", "AQ-07 AQ-123")
# ['AQ-07']
```

ASSEMBLER BASE

Du journal brut à `mesures.csv`




```
conn = sqlite3.connect("capteurs.db")
quartier_de = dict(conn.execute(
    "SELECT id, quartier FROM capteurs"))
conn.close()

rejets = Counter()
with open("journal-capteurs.log", encoding="utf-8") as src, \
     open("mesures.csv", "w", encoding="utf-8", newline="") as dst:
    w = csv.writer(dst)
    w.writerow(["horodatage", "capteur", "quartier",
                "no2", "pm10", "temp"])
    for ligne in src:
        m = MESURE.match(ligne.rstrip("\n"))
        if m is None:
            rejets["ERROR" if " ERROR " in ligne
                   else "illisible"] += 1
            continue
        d = m.groupdict()
        w.writerow([d["horodatage"], d["capteur"],
                    quartier_de[d["capteur"]],
                    d["no2"], d["pm10"], d["temp"]])

print(rejets)   # Counter({'ERROR': 1, 'illisible': 1})
```

TRAVAUX PRATIQUES 2B

Du journal des capteurs à `mesures.csv`

≈ 50 min · Individuel · Restitution 5 min · `tp/tp2b-logs-sqlite-csv/`

1. **Interroger la base**

   Se connecter à `capteurs.db`, compter les capteurs par quartier, construire `quartier_de`.

2. **Écrire la regex**

   Tester le motif sur les lignes d'exemple de l'énoncé avant de l'appliquer au journal entier.

3. **Écrire `mesures.csv`**

   Une ligne par mesure lue, avec le quartier, les valeurs manquantes laissées vides.

4. **Compter les rejets, par cause**

   Erreurs de capteur, lignes illisibles, capteur inconnu de la base (`dict.get`, ou `try` / `except KeyError` vu hier, qui **compte** au lieu d'avaler). Le total doit retomber sur le nombre de lignes du journal.

Les fichiers

`enonce.md` · `depart.py`
`capteurs.db` · `journal-capteurs.log`

Bonus

La classe `Capteur` du TP 2a, sur le journal entier, et la liste des capteurs en alerte NO_2, triée, écrite dans `alertes.txt` et dans une table `alertes` de la base.

CONTRÔLE DES ACQUIS

Cinq questions avant de fermer le module 2

Q1 Deux capteurs créés séparément partagent soudain les mêmes mesures. Où chercher ?

Une liste déclarée en attribut de classe au lieu d'être créée dans `__init__` : il n'en existe qu'une, partagée par toutes les instances.

Q2 Un script lit « HÃ´tel de Ville » au lieu de « Hôtel de Ville ». Que s'est-il passé ?

Un fichier UTF-8 lu avec l'encodage du système (cp1252 sous Windows). Il manque `encoding="utf-8"` à l'`open`.

Q3 Votre CSV, ouvert sous Windows, a une ligne vide sur deux. Pourquoi ?

Le fichier a été ouvert sans `newline=""` : le module `csv` écrit `\r\n`, le mode texte ajoute un `\r`.

Q4 Pourquoi `execute(sql, (quartier,))` plutôt qu'une f-string ?

La valeur est transmise à part, elle ne peut pas modifier la requête : pas d'injection SQL, et les apostrophes ne cassent rien.

Q5 `re.match(r"AQ-\d\d", ligne)` rend `None` alors que la ligne contient AQ-07. Pourquoi ?

`match` ne cherche qu'au début de la chaîne. Il faut `re.search`, ou un motif qui décrit le début de la ligne.

CONTRÔLE DES ACQUIS

Le quiz en ligne du module 2

Il porte sur ce que le module vient de couvrir : classes, fichiers texte, `csv`, `sqlite3` et expressions régulières.

```
wytasoft.com/quizz/python-traitement-donnees-module-2-fr
```

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

ANNEXES DU MODULE 2 AVANCÉ

Écrire moins de code

Les classes de données, et les générateurs pour lire un journal sans fin.

`@dataclass` : `__init__`, `__repr__` et `__eq__` écrits pour vous

```python
from dataclasses import dataclass, field

@dataclass(order=True)
class Capteur:
    id: str
    quartier: str
    mesures: list[float] = field(default_factory=list)

c = Capteur("AQ-07", "Port")
c          # Capteur(id='AQ-07', quartier='Port', mesures=[])
c == Capteur("AQ-07", "Port")     # True

@dataclass(frozen=True)
class Mesure:
    capteur: str
    no2: float

m = Mesure("AQ-07", 41.2)
m.no2 = 0      # FrozenInstanceError: cannot assign to field 'no2'
{m}            # {Mesure(capteur='AQ-07', no2=41.2)}
```

On déclare les champs avec leur type ; le décorateur écrit `__init__`, `__repr__`, `__eq__`, et avec `order=True` les comparaisons.

`frozen=True` rend l'instance immuable, donc hachable : utilisable dans un ensemble ou comme clé.

> Le piège de la liste partagée est **refusé** : `mesures: list = []` lève `ValueError: mutable default … use default_factory`.

GÉNÉRATEURS AVANCÉ

`yield` : une fonction qui produit au fil de l'eau

```
def capteurs_lus(chemin):
    with open(chemin, encoding="utf-8") as f:
        for ligne in f:
            if " INFO " in ligne:
                yield ligne.split()[2]

g = capteurs_lus("journal-capteurs.log")
g          # <generator object capteurs_lus at 0x ... >
next(g)    # 'AQ-07'
list(g)    # ['AQ-01', 'AQ-10', 'AQ-04', 'AQ-07']
```

Une fonction qui contient `yield` ne s'exécute pas à l'appel : elle rend un **générateur**, qui avance d'un `yield` à chaque demande.

Rien n'est stocké : on peut ainsi séparer la **lecture** (un générateur de mesures) du **traitement** (la boucle qui écrit le CSV), sans charger le journal.

> Un générateur ne se parcourt **qu'une fois**. Après `list(g)`, il est vide : `next(g)` lève `StopIteration`.

MODULE 3

L'écosystème Python scientifique

Les bibliothèques incontournables, Anaconda, les environnements virtuels et Jupyter.

JOUR 2 · MODULE 3

L'écosystème Python scientifique

Les librairies qui comptent, les outils pour les installer sans casser son poste, et un environnement de travail qu'on peut reproduire.

DURÉE

≈ 2 h 30 d'exposé + 2 TP de 30 min

À LA FIN

Un environnement isolé, exporté, et un notebook qui lit `mesures.csv`

TP

3a : l'environnement · 3b : Jupyter et formats

FEUILLE DE ROUTE

Ce que couvre le module 3

1 **Le panorama**
Les librairies incontournables, les outils, et comment juger une librairie qu'on ne connaît pas.

2 **Distribution et environnements**
Anaconda et ses cousins, `venv`, `conda`, installer une librairie, exporter et dupliquer.

TP **TP 3a : l'environnement `aircite`**
Créer, vérifier, exporter, dupliquer, déclarer à Jupyter.

3 **IPython et Jupyter**
L'interpréteur amélioré, le notebook, son piège principal, et comment le partager.

4 **Démarrer un projet, choisir un format**
L'arborescence, les bonnes pratiques, CSV, Parquet, HDF5 et les autres.

TP **TP 3b : Jupyter et formats**
Le notebook sur le bon noyau, `mesures.csv` en quatre formats, l'export HTML.

La question de l'après-midi

Si je donne mon notebook à un collègue, **que doit-il installer, et comment être sûr qu'il obtiendra les mêmes chiffres ?**
Un résultat qu'on ne peut pas refaire tourner ailleurs n'est pas un résultat, c'est une anecdote.

SÉQUENCE 1

Le panorama

Une pile de librairies qui reposent toutes sur la même fondation, et des critères pour trier les autres.

PANORAMA BASE

Une pile, avec NumPy à la base



Polars, DuckDB, Dask : des voisins qui parlent le même format (Arrow, NumPy)

Chaque étage **utilise** celui du dessous : un DataFrame pandas range ses colonnes dans des tableaux NumPy, Matplotlib trace des tableaux NumPy, scikit-learn attend des tableaux NumPy.

> Apprendre NumPy, c'est apprendre la langue commune. Le reste de la pile se lit ensuite beaucoup plus vite.

On appelait historiquement « SciPy Stack » l'ensemble NumPy, SciPy, Matplotlib, pandas, IPython : c'est le titre du module 4.

PANORAMA BASE

Qui fait quoi

LIBRAIRIE	CE QU'ELLE FAIT	SUR AIR'CITÉ
NumPy	Tableaux à n dimensions, calcul vectoriel, algèbre linéaire	Une image vue comme une matrice de pixels
pandas	Tables étiquetées : lire, filtrer, regrouper, pivoter	La moyenne de NO_2 par quartier et par heure
Matplotlib	Tracés 2D, du brouillon à la figure publiée	La courbe d'une journée, la carte des capteurs
SciPy	Statistiques, optimisation, interpolation, signal, géométrie	La transformée de Fourier qui révèle le cycle de 24 h
scikit-learn	Apprentissage automatique classique	Hors programme : prédire un pic de pollution
seaborn · **Plotly**	Graphiques statistiques prêts à l'emploi · graphiques interactifs	Une carte de chaleur heure × jour, en une ligne
statsmodels	Modèles statistiques, tests, séries temporelles	Une régression NO_2 ~ température

OUTILS BASE

Les outils open source du quotidien

Pour écrire et explorer

- **Jupyter** (Notebook, JupyterLab) : le carnet de calcul, séquence 3
- **VS Code** : éditeur généraliste, ouvre aussi les notebooks
- **Spyder** : l'IDE scientifique, explorateur de variables à la MATLAB

Pour installer et isoler

- **pip** et **venv** : livrés avec Python
- **conda**, **mamba** : paquets et environnements, au-delà de Python
- **uv** : un remplaçant récent et très rapide de pip et venv

Pour versionner et interroger

- **Git** : l'historique du code et des notebooks
- **DuckDB** : du SQL sur des CSV et des Parquet, sans serveur
- **SQLite** : la base dans un fichier, vue au module 2

Pour montrer ses résultats : Streamlit et Dash (une application web en quelques lignes de Python), Apache Superset et Metabase (tableaux de bord branchés sur une base, sans code).

> Aucun de ces outils n'est obligatoire. Le minimum vital pour la suite : **un environnement isolé, Jupyter, et Git**.

TROUVER BASE

Où chercher une librairie

PyPI pypi.org	Le dépôt officiel de Python, celui de `pip install`. Tout y est, le meilleur comme l'abandonné.
conda-forge conda-forge.org	Le dépôt communautaire de conda : des paquets compilés, testés ensemble, gratuits pour tous.
GitHub	Le code source, les tickets, la vie du projet. C'est là qu'on juge la santé d'une librairie.
Listes de référence	*awesome-python*, les projets parrainés par NumFOCUS, scientific-python.org : une première sélection faite par d'autres.

Un nom ressemblant n'est pas le bon

Des paquets malveillants imitent des noms connus sur PyPI, à une lettre près. Copier le nom depuis la documentation officielle, ne jamais le taper de mémoire.

JUGER BASE

Sept questions avant d'adopter une librairie

1. **Dernière version publiée ?**

   Moins d'un an : vivante. Plus de deux ans : à vérifier.

2. **Combien de mainteneurs ?**

   Un seul, c'est un projet qui s'arrête le jour où il change de travail.

3. **Quelle licence ?**

   MIT, BSD, Apache : sans contrainte. GPL : à faire valider avant un usage commercial.

4. **Les tickets reçoivent-ils une réponse ?**

   Des centaines de tickets sans réponse disent plus qu'un beau README.

5. **Compatible avec votre Python ?**

   Les grandes librairies suivent la règle SPEC 0 : une version de Python reste prise en charge environ trois ans.

6. **Qui l'utilise ?**

   Téléchargements, dépendants sur GitHub, présence sur conda-forge.

7. **Une fondation derrière ?**

   NumFOCUS, Linux Foundation, une entreprise : un filet, pas une garantie.

SÉQUENCE 2

Distribution et environnements

Installer des librairies compilées sans douleur, et isoler chaque projet dans sa propre boîte.

DISTRIBUTION BASE

Pourquoi une distribution scientifique

Le problème

NumPy, SciPy ou pandas ne sont pas que du Python : ce sont des milliers de lignes de **C, C++ et Fortran** à compiler, liées à des bibliothèques de calcul (BLAS, LAPACK).

Les compiler soi-même sous Windows demande un compilateur, des en-têtes, des heures. Personne ne le fait.

Ce qu'apporte une distribution

- Des **binaires déjà compilés** pour votre système
- Une bibliothèque de calcul optimisée : **MKL** d'Intel ou **OpenBLAS**
- Des paquets **non Python** : GDAL, HDF5, le compilateur, R
- Un **solveur** qui choisit des versions compatibles entre elles

> Depuis que pip installe des binaires précompilés (les *wheels*), `pip install numpy` marche aussi. conda garde l'avantage dès qu'une dépendance n'est **pas** écrite en Python.

DISTRIBUTION BASE

Anaconda, Miniconda, Miniforge

	CONTENU	DÉPÔT PAR DÉFAUT	GRATUIT EN ENTREPRISE ?
Anaconda	conda, Python, des centaines de paquets, Navigator, Spyder, Jupyter. Plusieurs Go.	Celui d'Anaconda, `defaults`	Non à partir de 200 personnes
Miniconda	conda et Python, rien d'autre. On installe ce qu'on veut.	Celui d'Anaconda, `defaults`	Même règle : c'est le dépôt qui compte
Miniforge	conda, mamba et Python, rien d'autre.	`conda-forge`, communautaire	Oui

> Les conditions d'Anaconda rendent payant l'usage de **son dépôt** par les organisations de 200 personnes ou plus. Avant d'installer Anaconda au travail, **vérifiez la licence de votre organisation**. Miniforge évite la question.

Pourquoi un environnement par projet

env « rapport-2023 »

- Python 3.10
- pandas 1.5
- numpy 1.26

env « aircite »

- Python 3.12
- pandas 2.3
- numpy 2.3

Le Python du système : on n'y installe rien

Un environnement, c'est **un dossier** qui contient son propre Python et ses propres librairies.

- Deux projets peuvent exiger deux versions incompatibles
- Mettre à jour l'un ne casse pas l'autre
- On sait **exactement** ce dont le projet dépend
- On le jette et on le recrée sans risque

L'outil livré avec Python : venv

Créer et activer

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1
# Windows (invite de commandes)
.venv\Scripts\activate.bat
# Linux, macOS
source .venv/bin/activate
```

Installer, figer

```
python -m pip install pandas matplotlib
python -m pip freeze > requirements.txt
```

Vérifier où l'on est

```
python -c "import sys; print(sys.prefix)"
# C:\projets\aircite\.venv
```

Recréer ailleurs

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

> venv isole les librairies, **pas la version de Python** : il réutilise celui qui l'a créé.

CONDA BASE

Les environnements conda

```
conda create -n aircite python=3.12
conda activate aircite

conda install -c conda-forge pandas matplotlib scipy

conda env list          # tous les environnements
conda list              # le contenu de celui-ci
conda deactivate
conda env remove -n aircite
```

Ce que conda ajoute à venv

Il installe **la version de Python** demandée, et des paquets qui ne sont pas du Python. Les environnements sont rangés dans un dossier central, appelés par leur nom.

pip dans conda : en dernier

Installer d'abord tout ce qui existe en conda, puis le reste avec `pip`. Une fois pip passé, un nouveau `conda install` peut défaire ce qu'il a fait.

INSTALLER BASE

Installer, inspecter, mettre à jour une librairie

Avec pip

```
python -m pip install pandas            # la dernière
python -m pip install "pandas==2.3.3"   # exactement
python -m pip install "pandas≥2.2,<3"   # une plage
python -m pip install --upgrade pandas
python -m pip uninstall pandas
python -m pip list                       # tout
python -m pip show pandas                # une seule
```

```
Name: pandas
Version: 2.3.3
License: BSD 3-Clause License
Requires: numpy, python-dateutil, pytz, tzdata
```

Avec conda

```
conda install -c conda-forge pandas=2.3
conda update pandas
conda remove pandas
conda search -c conda-forge pandas
```

1. **Toujours dans l'environnement activé**

   Sinon, la librairie part dans le Python du système.

2. **conda d'abord, pip en dernier**

   Et un seul canal par environnement : `conda-forge` ou `defaults`, pas les deux mélangés.

3. **Guillemets autour des contraintes**

   Sans eux, `>` devient une redirection du terminal.

DÉCRIRE BASE

requirements.txt ou environment.yml

`requirements.txt` · pip

```
# après un seul pip install pandas==2.3.3
numpy==2.4.6
pandas==2.3.3
python-dateutil==2.9.0.post0
pytz==2026.5
six==1.17.0
tzdata==2026.5
```

`pip freeze` fige **tout**, dépendances comprises. Ne décrit **pas** la version de Python.

`environment.yml` · conda

```
name: aircite
channels:
  - conda-forge
dependencies:
  - python=3.12
  - pandas=2.3
  - matplotlib
  - scipy
  - jupyterlab
  - pip
  - pip:
      - une-librairie-absente-de-conda
```

Le nom, le canal, Python lui-même, et une section `pip`.

EXPORTER ET DUPLIQUER BASE

Exporter un environnement, le dupliquer

COMMANDE	CE QU'ELLE PRODUIT	POUR
`conda env export > env.yml`	Tout, versions et numéros de build compris, avec la section pip	Refaire à l'identique, **même système**
`conda env export --from-history`	Seulement ce que vous avez demandé explicitement	Partager entre Windows, Linux, macOS
`conda create -n copie --clone aircite`	Une copie locale de l'environnement	Essayer une mise à jour sans risque
`conda env create -f env.yml`	L'environnement recréé à partir du fichier	Le poste du collègue, le serveur
`python -m pip freeze > requirements.txt`	Toutes les librairies pip, versions figées	L'équivalent côté venv

> `--from-history` **oublie ce que pip a installé**. Relire le fichier exporté avant de le partager.

NOYAU JUPYTER BASE

Brancher l'environnement sur Jupyter

```
# dans l'environnement activé
python -m pip install ipykernel        # ou conda install ipykernel
python -m ipykernel install --user --name aircite --display-name "Python (aircite)"

jupyter kernelspec list                # les noyaux connus
```

Le notebook propose ensuite **Python (aircite)** dans la liste des noyaux.

Le test qui ne ment pas

Dans une cellule :

```
import sys
sys.executable
```

Le chemin doit pointer dans `aircite`. Sinon, le notebook tourne ailleurs.

TRAVAUX PRATIQUES 3A

L'environnement `aircite`

≈ 30 min · Individuel · Restitution 5 min · Voie conda ou voie venv

1. **Créer l'environnement `aircite`**

   Depuis `environment.yml` ou `requirements.txt` : pandas 2.x, matplotlib, scipy, jupyterlab, ipykernel, pyarrow, openpyxl, duckdb. Lancer la création tout de suite.

2. **Le vérifier**

   `python verifier_env.py` : « Environnement prêt », et un préfixe qui contient `aircite` ou `.venv`.

3. **L'exporter**

   `conda env export` complet et `--from-history`, ou `pip freeze` : comparer les fichiers.

4. **Le dupliquer**

   Un clone, y changer une version, constater que l'original n'a pas bougé.

5. **Le déclarer à Jupyter**

   `python -m ipykernel install --user --name aircite`, puis `jupyter kernelspec list`.

Les fichiers

`tp/tp3a-environnement/`

`enonce.md` · `environment.yml`

`requirements.txt` · `verifier_env.py`

Un repère

8 librairies demandées dans `requirements.txt`, 105 lignes dans le `pip freeze` qui en sort.

SÉQUENCE 3

IPython et Jupyter

Un interpréteur qui aide, un carnet qui mélange code, résultats et texte, et le piège qui va avec.

IPYTHON BASE

IPython, l'interpréteur qui aide

S'informer sans quitter la console

```
import pandas as pd
pd.read_csv?        # la documentation
pd.read_csv??       # le code source
pd.read_<Tab>       # la complétion
```

Les commandes magiques

```
%run nettoyage.py   # exécute un script, garde ses variables
%who                # les variables définies
%history -n 1-5     # les cinq premières commandes
%pip install seaborn  # pip, dans CE noyau
```

Mesurer avec `%timeit`

```
import numpy as np
valeurs = list(range(1_000_000))
tableau = np.arange(1_000_000)

%timeit sum(valeurs)
# 44.8 ms ± 7.33 ms per loop ( ... )
%timeit tableau.sum()
# 1.32 ms ± 519 µs per loop ( ... )
```

> Le même total, **plusieurs dizaines de fois plus vite** : c'est NumPy, et c'est le module 4.

SE DOCUMENTER SI LE TEMPS LE PERMET

Où chercher quand on bloque

Sans quitter Python

```
help(len)
```

```
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.
```

```
pd.read_csv?      # dans IPython ou Jupyter
dir(df)           # les méthodes d'un objet
```

`Maj + Tab` dans Jupyter affiche la signature de la fonction sous le curseur.

La doc officielle	docs.python.org (en français), numpy.org, pandas.pydata.org : les guides de démarrage, puis la référence
Les antisèches	Les *cheat sheets* pandas et NumPy, une page à imprimer
Stack Overflow	Coller la **dernière ligne** de l'erreur dans le moteur de recherche
Un assistant IA	Rapide pour un premier jet. Relire et **exécuter** chaque ligne : il invente parfois des fonctions

Un serveur, un noyau, un navigateur



Fermer l'onglet n'arrête pas le noyau : les variables restent en mémoire.
Redémarrer le noyau les efface toutes.

```
jupyter lab        # ou : jupyter notebook
```

Le serveur ouvre le navigateur sur `http://localhost:8888`. Tout reste sur votre poste.

Le fichier `.ipynb` est du **JSON** : le code, le texte, **et les résultats** de la dernière exécution.

JUPYTER BASE

Notebook, JupyterLab, et les cellules

Notebook	Un notebook par onglet, interface dépouillée. Depuis la version 7, bâti sur les briques de JupyterLab.
JupyterLab	Un espace de travail : explorateur de fichiers, plusieurs notebooks, terminal, visionneuse de CSV.
VS Code	Ouvre les mêmes `.ipynb`, avec le débogueur de l'éditeur.

Le fichier est le même dans les trois : on change d'outil sans rien convertir.

Les raccourcis qui servent

`Maj + Entrée`	Exécuter, passer à la suivante
`Échap` puis `A` / `B`	Nouvelle cellule au-dessus / en dessous
`Échap` puis `M` / `Y`	Cellule de texte (Markdown) / de code
`Échap` puis `D D`	Supprimer la cellule
`Échap` puis `0 0`	Redémarrer le noyau

LE PIÈGE BASE

L'ordre d'exécution n'est pas l'ordre des cellules

Ce qui s'est passé

```
[1]  df = pd.read_csv("mesures.csv")
[3]  df["no2"] = df["no2"] / 1000   # µg → mg
[4]  df["no2"].mean()
# mille fois trop petit, sans aucune erreur
```

Les numéros `[ ]` trahissent l'ordre réel : où est le `[2]` ? La deuxième cellule a été lancée deux fois, en `[2]` puis en `[3]` : la colonne a été divisée deux fois.

La règle

Avant de conclure, avant de partager :

Noyau → Redémarrer et tout exécuter.

Si le notebook ne passe pas de haut en bas sur un noyau neuf, son résultat ne vaut rien.

Et écrire des cellules qu'on peut relancer sans dommage : créer une nouvelle colonne plutôt que d'écraser l'ancienne.

PARTAGER BASE

Documenter, exporter, versionner un notebook

Une cellule Markdown

```
# Air'Cité : NO2 par quartier
Source : `data/processed/mesures.csv`,
produit par le TP 2b.

- **unité** : µg/m³
- lignes rejetées : voir `rapport.txt`
```

Exporter

```
jupyter nbconvert --to html analyse.ipynb
jupyter nbconvert --to script analyse.ipynb
jupyter nbconvert --to html --execute analyse.ipynb
```

`--to html`	Une page autonome, résultats et graphiques compris : pour qui n'a pas Python
`--to script`	Un `.py` avec le code des cellules : le début d'un vrai script
`--execute`	Réexécute tout sur un noyau neuf avant d'exporter : la règle de la slide précédente, automatisée

Git et les notebooks

Le `.ipynb` est du JSON qui contient les sorties : chaque exécution change le fichier. `nbstripout` retire les sorties avant le commit, `jupytext` tient une copie `.py` lisible.

SÉQUENCE 4

Démarrer un projet, choisir un format

Ranger un projet pour qu'un autre s'y retrouve, et savoir dans quoi stocker ses données.

BONNES PRATIQUES BASE

Une arborescence, des règles simples

```
aircite/
├── README.md           # quoi, comment lancer
├── environment.yml     # ou requirements.txt
├── .gitignore
├── data/
│   ├── raw/            # brut, jamais modifié
│   └── processed/      # produit par le code
├── notebooks/
│   └── 01-exploration.ipynb
├── src/aircite/        # le code réutilisé
│   └── nettoyage.py
└── reports/figures/
```

1. **Les données brutes sont en lecture seule**

   Tout ce qui est dérivé se régénère depuis `data/raw` par le code.

2. **L'environnement est décrit dans un fichier**

   Versionné avec le code, séquence 2.

3. **Le code qui se répète quitte le notebook**

   Une fonction dans `src/`, importée, testable : le module 1 l'a montré.

4. **Git pour le code, pas pour les données**

   `data/` et `.venv/` dans `.gitignore`.

5. **Un notebook partagé passe de haut en bas**

   Sur un noyau neuf, séquence 3.

Le même tableau, cinq formats

FORMAT	TAILLE	LECTURE	CE QU'IL GARDE, CE QU'IL PERD
CSV	1,23 Mo	32 ms	Lisible partout. Perd les types : l'horodatage revient en **texte**
Parquet	0,12 Mo	13 ms	Compressé, colonnaire, garde les types ; lit 2 colonnes sans lire les autres
JSON (lignes)	2,61 Mo	≈ 0,4 s	Fait pour l'échange web. L'horodatage revient en **entier** (millisecondes)
Excel `.xlsx`	0,79 Mo	≈ 10 s	Ce que les métiers ouvrent. Lent à lire, limité à 1 048 576 lignes
pickle	1,17 Mo	15 ms	Fidèle et rapide, mais propre à Python et **dangereux** à ouvrir

25 920 lignes (12 capteurs, 90 jours, une mesure par heure), écrites et relues avec pandas 2.3, pyarrow et openpyxl.

FORMATS SCIENTIFIQUES BASE

Quand les données ne sont pas une table

FORMAT	POUR QUOI	AVEC
.npy · .npz	Un ou plusieurs tableaux NumPy, tels quels	`np.save`, `np.load`
HDF5 `.h5`	Un « système de fichiers dans un fichier » : gros tableaux, groupes, métadonnées	`h5py`, PyTables (`pd.read_hdf`)
NetCDF `.nc`	Grilles multidimensionnelles étiquetées : climat, océan, météo	`xarray`, `netCDF4`
FITS	Images et tables de l'astronomie	`astropy.io.fits`
GeoJSON · Shapefile	Points, lignes, polygones géographiques	`json`, GeoPandas
Images PNG, TIFF	Des matrices de pixels	Matplotlib, Pillow, scikit-image

> Le point commun : à la lecture, tout **devient un tableau NumPy**. Retour à la base de la pile.

SÉCURITÉ SI LE TEMPS LE PERMET

pickle : n'ouvrez que ce que vous avez écrit

```
df.to_pickle("mesures.pkl")
pd.read_pickle("mesures.pkl")
```

pickle enregistre n'importe quel objet Python, et sait le reconstruire. Pour le reconstruire, il **exécute des instructions** contenues dans le fichier.

> Un fichier pickle reçu peut exécuter n'importe quel code à l'ouverture. La documentation de Python le dit en rouge.

Ce qu'on en fait

- Un cache personnel, sur son poste : oui
- Un fichier reçu par courriel, téléchargé : **jamais**
- Un format d'échange ou d'archive : non, Parquet

`np.load` refuse par défaut les tableaux qui contiennent du pickle (`allow_pickle=False`) : ne pas le désactiver sans savoir pourquoi.

TRAVAUX PRATIQUES 3B

Jupyter et formats

≈ 30 min · Individuel · Restitution 5 min · Noyau Python (aircite)

1. **Le bon noyau**

   `jupyter lab`, noyau **Python (aircite)**, puis `%run ../tp3a-environnement/verifier_env.py`.

2. **Markdown et IPython**

   Un en-tête : sujet, source, unités. `pd.read_csv?`, et `%timeit` d'une somme, en Python pur puis avec NumPy.

3. **Lire `mesures.csv`**

   Le fichier du TP 2b, sans puis avec `parse_dates` : le type de `horodatage`.

4. **Trois autres formats**

   Parquet, Excel, JSON : écrire, relire, comparer la taille et le type de l'horodatage.

5. **Avant de partager**

   « Restart Kernel and Run All Cells », puis `jupyter nbconvert --to html`.

Les fichiers

`tp/tp3b-jupyter-formats/`

`enonce.md` · `depart-3b.ipynb`

Votre `mesures.csv`, ou celui de `tp/donnees/`

Bonus

L'arborescence de la séquence 4 dans un dépôt Git, avec son `.gitignore`.

CONTRÔLE DES ACQUIS

Cinq questions avant de fermer le module 3

Q1 Votre entreprise compte 3 000 salariés. Pouvez-vous installer Anaconda librement sur votre poste ?

Pas sans vérifier : l'usage du dépôt d'Anaconda est payant à partir de 200 personnes. Miniforge, sur conda-forge, évite la question.

Q2 Vous faites `pip install seaborn` et le notebook répond `ModuleNotFoundError`. Deux causes probables ?

L'environnement n'était pas activé au moment du `pip install`, ou le notebook tourne sur un autre noyau. `sys.executable` tranche ; `%pip` installe au bon endroit.

Q3 Quel export conda partager avec un collègue sous Linux, quand vous êtes sous Windows ?

`conda env export --from-history` : seulement ce que vous avez demandé, sans les builds propres à Windows. En vérifiant qu'il n'oublie pas les paquets pip.

Q4 Votre notebook donne le bon résultat. Que faire avant de l'envoyer ?

Redémarrer le noyau et tout exécuter de haut en bas. Si ça ne passe pas sur un noyau neuf, le résultat dépend d'un état disparu.

Q5 CSV ou Parquet pour stocker les mesures nettoyées que vous relirez chaque jour ?

Parquet : plus petit, plus rapide, et il garde les types, dont l'horodatage. Le CSV reste pour l'échange.

CONTRÔLE DES ACQUIS

Le quiz en ligne du module 3

Il porte sur ce que le module vient de couvrir : bibliothèques, Anaconda, environnements virtuels, Jupyter, formats de fichiers.

```
wytasoft.com/quizz/python-traitement-donnees-module-3-fr
```

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

ANNEXES DU MODULE 3 AVANCÉ

Les outils qui montent

Un gestionnaire d'environnements nouvelle génération, et du SQL ou du Polars directement sur vos fichiers.

GESTIONNAIRES AVANCÉ

uv, pixi, mamba : la nouvelle génération

uv

Écrit en Rust par Astral. Remplace pip, venv et la gestion des versions de Python, en beaucoup plus rapide. Paquets PyPI uniquement.

```
uv venv
uv pip install pandas
```

pixi

Par prefix.dev, sur les paquets conda-forge. Un environnement par projet, décrit dans un fichier, avec un fichier de verrouillage.

```
pixi init
pixi add pandas
```

mamba

Le même usage que conda, en plus rapide. Livré avec Miniforge ; son solveur est celui de conda par défaut depuis 2023.

```
mamba create -n aircite pandas
```

> Les idées ne changent pas : **un environnement par projet, décrit dans un fichier, recréable**. Seuls les outils vont plus vite.

INTERROGER AVANCÉ

Du SQL directement sur un CSV avec DuckDB

```
import duckdb

duckdb.sql("""
    SELECT quartier,
           round(avg(no2), 1) AS no2_moyen,
           count(*) AS n
    FROM 'mesures.csv'
    GROUP BY quartier
    ORDER BY no2_moyen DESC
""")
```

Aucun chargement préalable : DuckDB lit le fichier, devine les types, et n'en lit que ce dont la requête a besoin.

Quand c'est le bon outil

- On pense en SQL plutôt qu'en pandas
- Le fichier est plus gros que la mémoire
- On interroge des dizaines de Parquet d'un coup : `FROM 'data/*.parquet'`

`.df()` en fin de requête rend un DataFrame pandas : les deux mondes se parlent.

MODULE 4

La SciPy Stack

NumPy, pandas, Matplotlib et SciPy, sur les mesures d'Air'Cité : calculer, résumer, tracer, cartographier.

WYTASOFT
Unlock the power of your data

JOUR 3 · MODULE 4

La SciPy Stack
NumPy, Matplotlib, pandas, SciPy

Calculer, tracer, analyser : les quatre bibliothèques sur lesquelles repose tout le Python scientifique.

DURÉE

La journée · ≈ 4 h 45 d'exposé + 3 TP (2 h 10)

À LA FIN

Passer d'un CSV brut à des chiffres, une carte et un spectre

TP

4a images · 4b statistiques · 4c carte et Fourier

FEUILLE DE ROUTE

Ce que couvre le module 4

1. **La SciPy Stack**
   Le socle commun, et pourquoi tout repose sur NumPy.

2. **NumPy**
   Tableaux, vectorisation, broadcasting, types et `NaN`, fichiers, algèbre linéaire, images.

3. **Matplotlib · TP 4a**
   Figures et axes, tracés de base, habiller un graphique. TP : une image traitée avec NumPy.

4. **pandas · TP 4b**
   Lire, nettoyer, filtrer, regrouper, pivoter, joindre. TP : les statistiques d'Air'Cité.

5. **SciPy · TP 4c**
   Statistiques, géospatial, Fourier. TP : la carte et le spectre.

La question du jour

Le NO2 d'Air'Cité suit-il le trafic ? **À quelle heure, dans quel quartier, et avec quel rythme ?**
On y répondra trois fois : avec un tableau croisé, avec un test statistique, avec une transformée de Fourier.

Horaires

Matin : socle, NumPy, Matplotlib, TP 4a, et l'ouverture du CSV avec pandas. Après-midi : pandas, SciPy, TP 4b et 4c, bilan.

SÉQUENCE 1

La SciPy Stack, le socle

Quatre bibliothèques, une seule structure de données en dessous.

ARCHITECTURE BASE

Tout repose sur le tableau NumPy



NumPy fournit un seul objet, le tableau, et les opérations rapides dessus.

pandas, **SciPy** et **Matplotlib** le consomment et le produisent.

> Une colonne pandas, une image, un signal : **c'est un tableau NumPy**. Comprendre NumPy, c'est comprendre les trois autres.

LE MOT SI LE TEMPS LE PERMET

« SciPy Stack » : un nom plus qu'une norme

- **SciPy** est une bibliothèque, qu'on importe : `import scipy`.
- La **SciPy Stack** désignait l'ensemble NumPy, SciPy, Matplotlib, pandas, IPython, SymPy : ce qu'il fallait installer pour « faire du scientifique ».
- Le projet a longtemps publié une spécification de cet ensemble ; elle n'est **plus maintenue**. Le mot est resté dans l'usage et dans les programmes de formation.

Ce qui en tient lieu aujourd'hui

Les **distributions** (Anaconda, vue au module 3) et les fichiers d'environnement : c'est vous qui fixez la liste et les versions.

Les versions de ce support

NumPy 2.3 · pandas 2.3 · Matplotlib 3.11 · SciPy 1.17, sur Python 3.11.

SÉQUENCE 2

NumPy, le calcul sur des tableaux

Un type, une forme, et des opérations qui s'appliquent à tous les éléments d'un coup.

Le ndarray : un type, une forme

```python
import numpy as np

a = np.array([12.5, 31.0, 48.2, 27.9])
a           # array([12.5, 31. , 48.2, 27.9])
a.dtype     # dtype('float64')
a.shape     # (4,)
a.ndim      # 1

m = np.array([[1, 2, 3],
              [4, 5, 6]])
m.dtype     # dtype('int64')
m.shape     # (2, 3) : 2 lignes, 3 colonnes
m.size      # 6
```

	LISTE PYTHON	NDARRAY
Contenu	N'importe quoi, mélangé	**Un seul type** (`dtype`)
Taille	Variable	**Fixe** (`shape`)
`+`	Concatène	**Additionne** élément par élément
Calcul	Boucle Python	Boucle en C

> `[1, 2] + [3, 4]` donne `[1, 2, 3, 4]`.
> `np.array([1, 2]) + np.array([3, 4])` donne `array([4, 6])`.

CRÉER BASE

Créer des tableaux sans les taper

```
np.zeros((2, 3))
# array([[0., 0., 0.],
#        [0., 0., 0.]])

np.arange(0, 10, 2)
# array([0, 2, 4, 6, 8])

np.linspace(0, 1, 5)
# array([0.  , 0.25, 0.5 , 0.75, 1.  ])

np.arange(12).reshape(3, 4)
# array([[ 0,  1,  2,  3],
#        [ 4,  5,  6,  7],
#        [ 8,  9, 10, 11]])
```

`zeros`, `ones`, `full`	Remplir une forme donnée
`arange`	Comme `range`, pas entier ou décimal
`linspace`	*n* points régulièrement espacés, bornes incluses
`reshape`	Même données, autre forme
`rng.normal`, `rng.uniform`	Aléatoire, avec `rng = np.random.default_rng(42)`

La graine

`default_rng(42)` donne les mêmes tirages à chaque exécution : indispensable pour un résultat reproductible.

VECTORISER BASE

Une boucle de moins, cinquante fois plus vite

Boucle Python

```python
rng = np.random.default_rng(0)
no2 = rng.uniform(5, 80, 1_000_000)

r = []
for x in no2:
    r.append(x * 1.88)   # ppb → µg/m³
```

Vectorisé

```python
r = no2 * 1.88
```

Une seule expression : la boucle existe toujours, mais **en C**, sans interpréter une ligne Python par élément.

UN MILLION DE VALEURS	BOUCLE	VECTORISÉ	RAPPORT
Mesuré avec `timeit`, meilleur de 3	180 ms	3,7 ms	**≈ × 50**

> Le réflexe NumPy : **si vous écrivez `for` sur un tableau, cherchez l'opération qui le fait d'un coup.**

INDEXER BASE

Désigner des éléments : position, tranche, condition

```
v = np.array([18, 42, 7, 55, 31, 64])
v[0], v[-1]      # (18, 64)
v[1:4]           # array([42,  7, 55])
v[::2]           # array([18,  7, 31])

v > 40
# array([False, True, False, True, False, True])
v[v > 40]        # array([42, 55, 64])
(v > 40).sum()   # 3

np.where(v > 40, "alerte", "ok")
```

```
m = np.arange(12).reshape(3, 4)
m[1, 2]          # 6     ligne 1, colonne 2
m[:, 0]          # array([0, 4, 8])
m[1:, 2:]        # array([[ 6,  7],
                 #        [10, 11]])
```

Le masque booléen

`v > 40` est un tableau de `True` / `False`. Mis entre crochets, il **filtre**. Additionné, il **compte**.

> Combiner deux conditions : `(v > 20) & (v < 60)`, avec `&` et des parenthèses, jamais `and`.

PIÈGES BASE

Deux surprises NumPy, vues sur des données réelles

Une tranche est une vue

```
v = np.array([18, 42, 7, 55, 31, 64])
s = v[1:4]
s[0] = 0
v        # array([18,  0,  7, 55, 31, 64])

s = v[1:4].copy()   # indépendante
```

Le même piège qu'au module 1 avec les listes affectées par référence, mais **sur une tranche**, là où les listes Python copient.

Un octet déborde sans prévenir

```
p = np.array([1, 2, 3], dtype=np.uint8)
p - 2    # array([255,   0,   1], dtype=uint8)
```

Les pixels d'une image sont souvent en `uint8`, de 0 à 255. Sous zéro, on repart de 255.

La parade

Convertir avant de calculer : `p.astype(float) - 2`, ou travailler en flottants dès la lecture.

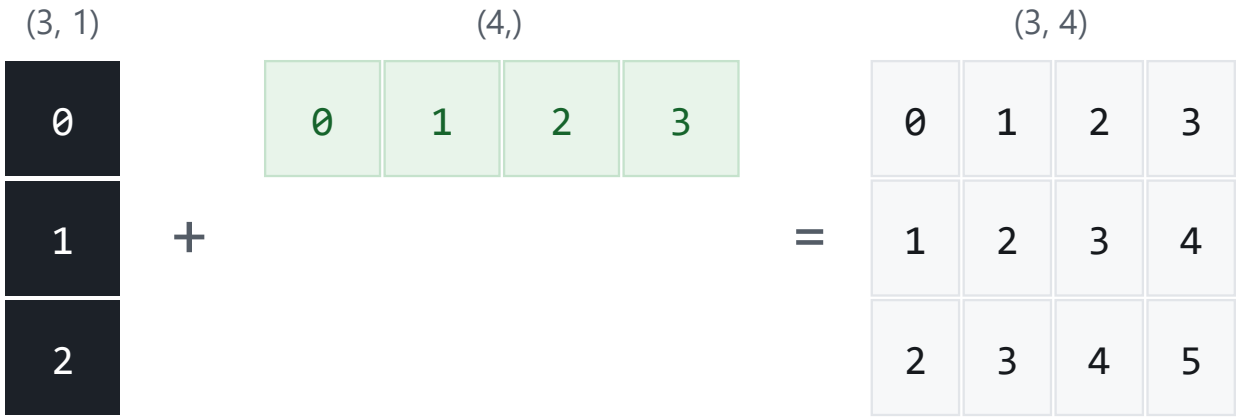
Opérer entre formes différentes

```python
mesures = np.array([[20, 35, 50],
                    [22, 30, 41]])

mesures - mesures.mean(axis=0)
# array([[-1. ,  2.5,  4.5],
#        [ 1. , -2.5, -4.5]])

np.arange(3).reshape(3, 1) + np.arange(4)
# array([[0, 1, 2, 3],
#        [1, 2, 3, 4],
#        [2, 3, 4, 5]])
```

Règle : on compare les formes **par la droite** ; une dimension vaut 1 ou est égale, sinon erreur.



La colonne est étirée sur 4 colonnes,
la ligne sur 3 lignes, sans rien recopier en mémoire.

AGRÉGER BASE

Résumer selon un axe

```python
# 3 capteurs (lignes) × 4 heures (colonnes)
x = np.array([[31, 44, 52, 29],
              [18, 22, 25, 17],
              [40, 61, 66, 38]])

x.sum()                    # 443
x.mean(axis=0)             # par heure
# array([29.67, 42.33, 47.67, 28.  ])
x.mean(axis=1).round(1)    # par capteur
# array([39. , 20.5, 51.2])
x.max(axis=1)              # array([52, 25, 66])
x.argmax(axis=1)           # array([2, 2, 2])
```

Comment retenir `axis`

`axis=0` fait **disparaître** les lignes : il reste une valeur par colonne.

`axis=1` fait disparaître les colonnes : il reste une valeur par ligne.

`sum` `mean` `std`	Somme, moyenne, écart-type
`min` `max`	Extrêmes
`argmin` `argmax`	**Position** de l'extrême
`np.percentile`	Quantiles, médiane

Convertir, compter les `NaN`, enregistrer

Convertir un type

```
t = np.array(["12.5", "31", "7.25"])   # du texte
f = t.astype(float)  # array([12.5 , 31.  ,  7.25])
f.astype(int)        # array([12, 31,  7])  tronqué
```

Une valeur manquante : `np.nan`

```
v = np.array([31.0, np.nan, 44.2, 18.5])
v.mean()             # np.float64(nan)
np.nanmean(v)        # np.float64(31.233333333333334)
np.isnan(v).sum()    # 1
```

Enregistrer et relire

```
m = np.arange(6).reshape(2, 3) * 1.5

np.save("m.npy", m)        # binaire, exact
np.load("m.npy")
# array([[0. , 1.5, 3. ],
#        [4.5, 6. , 7.5]])

np.savetxt("m.csv", m, delimiter=",", fmt="%.1f")
np.loadtxt("m.csv", delimiter=",")  # texte
```

`NaN` n'existe qu'en flottant

Un tableau d'entiers ne peut pas contenir de trou : `e[0] = np.nan` lève `ValueError`. Un trou force le passage en `float`.

ALGÈBRE LINÉAIRE BASE

Matrices, produit et systèmes

```
A = np.array([[2, 1],
              [1, 3]])
b = np.array([3, 5])

A * A        # élément par élément
A @ A        # produit matriciel
# array([[ 5,  5],
#        [ 5, 10]])
A.T          # transposée
np.linalg.det(A)   # 5.000000000000001

x = np.linalg.solve(A, b)
x            # array([0.8, 1.4])
A @ x        # array([3., 5.])  vérifié
```

Résoudre plutôt qu'inverser

Pour `A x = b`, `np.linalg.solve(A, b)` est plus rapide et plus précis que `np.linalg.inv(A) @ b`.

`*` n'est pas `@`

`*` multiplie case par case. Le produit matriciel s'écrit `@`.

Où cela sert : régression linéaire, changements de repère, conversion d'une image couleur en gris (slide suivante).

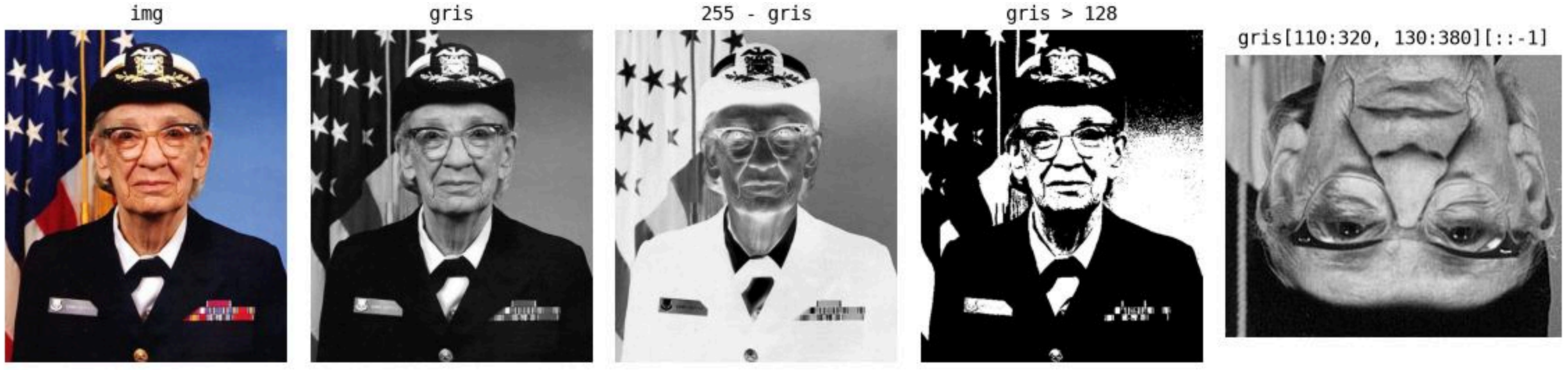
Une image est un tableau de nombres

```python
from matplotlib import cbook
img = plt.imread(cbook.get_sample_data("grace_hopper.jpg"))
img.shape     # (600, 512, 3)  hauteur, largeur, canaux R V B
img.dtype     # dtype('uint8')  entiers de 0 à 255
img[0, 0]     # array([21, 24, 77], dtype=uint8)  un pixel

gris = img @ np.array([0.299, 0.587, 0.114])
gris.shape    # (600, 512)  float64 : un canal, en flottants
```

```python
negatif = 255 - gris                # broadcasting
masque  = gris > 128                # booléens
masque.mean()                       # 0.285 : 28,5 % clairs
visage  = gris[110:320, 130:380]    # tranche (210, 250)
miroir  = gris[:, ::-1]             # gauche ↔ droite
petit   = gris[::2, ::2]            # (300, 256)

plt.imsave("gris.png", gris, cmap="gray")
```

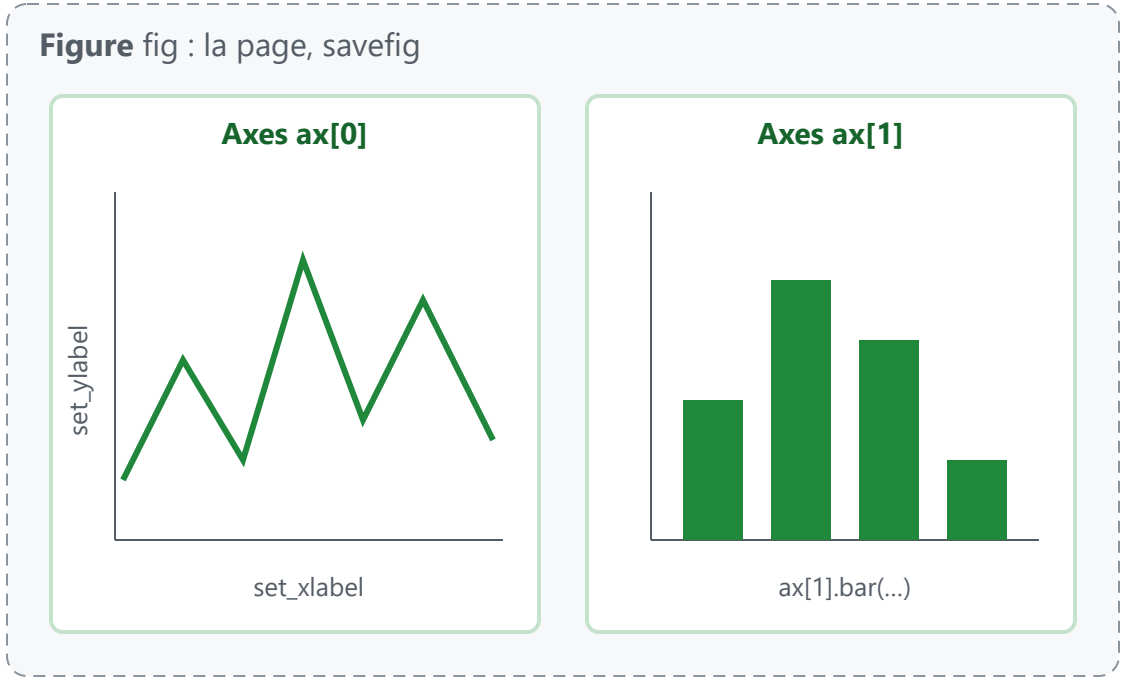


SÉQUENCE 3

Matplotlib, voir les données

Une figure, des axes, et quatre tracés qui suffisent à démarrer.

Une figure contient des axes




```python
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].plot(heures, no2)
ax[0].set_xlabel("heure")
ax[0].set_ylabel("NO2 (µg/m³)")
ax[0].set_title("AQ-01")

ax[1].bar(quartiers, moyennes)

fig.tight_layout()
fig.savefig("figure.png", dpi=150)
```

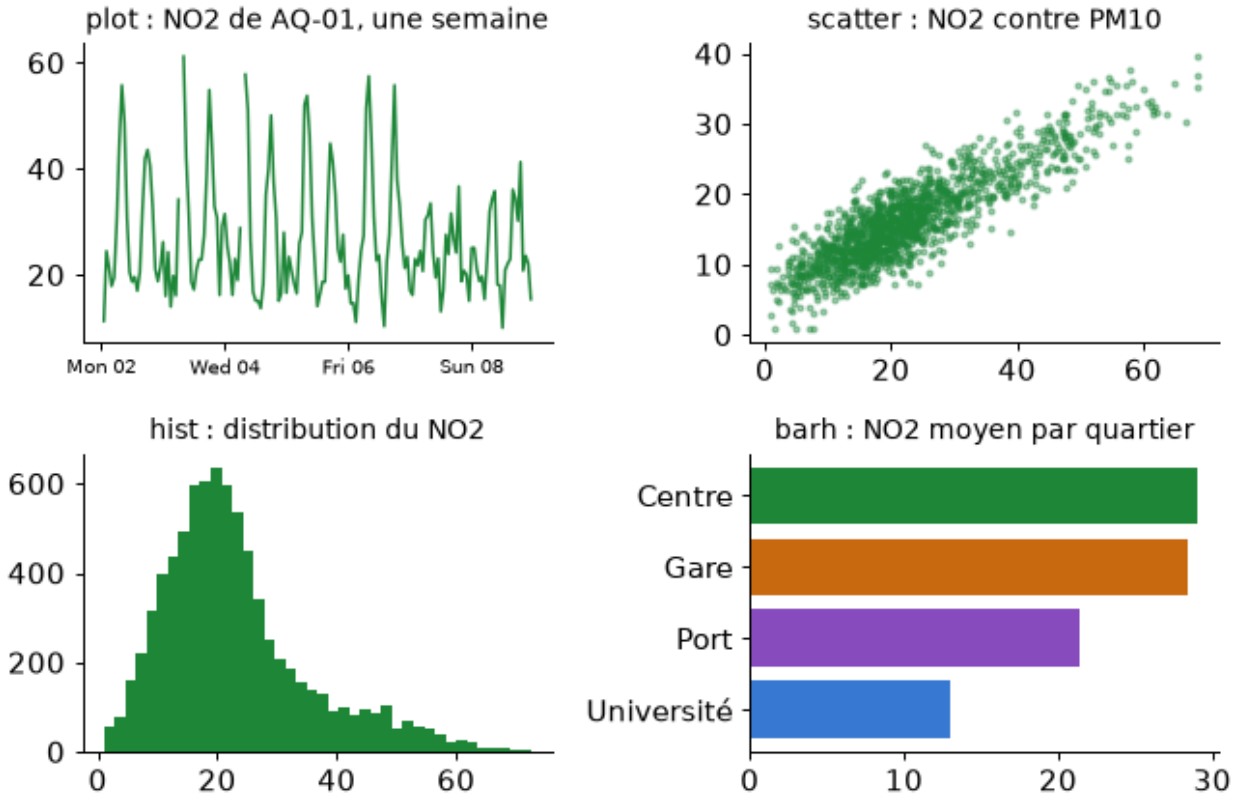
> On dessine **dans un axes**, on enregistre **la figure**.

Quatre tracés pour démarrer

MÉTHODE	POUR MONTRER
`ax.plot(x, y)`	Une évolution dans le temps
`ax.scatter(x, y)`	Le lien entre deux mesures
`ax.hist(v, bins=40)`	La distribution d'une mesure
`ax.bar(n, v)` `barh`	Comparer des catégories
`ax.imshow(t)`	Une image, une matrice

Choisir par la question

« Comment ça évolue ? » plot. « Est-ce lié ? » scatter. « Comment c'est réparti ? » hist. « Qui est le plus haut ? » bar.



Les vraies mesures d'Air'Cité, une ligne de code par vignette.

DEUX STYLES BASE

`plt.` ou `ax.` : l'interface d'état ou l'objet

pyplot : « la figure courante »

```
plt.plot(heures, no2)
plt.title("AQ-01")
plt.xlabel("heure")
plt.show()
```

Court, pratique pour un coup d'œil. Mais avec plusieurs graphiques, on ne sait plus qui est « courant ».

Orienté objet : on nomme ce qu'on dessine

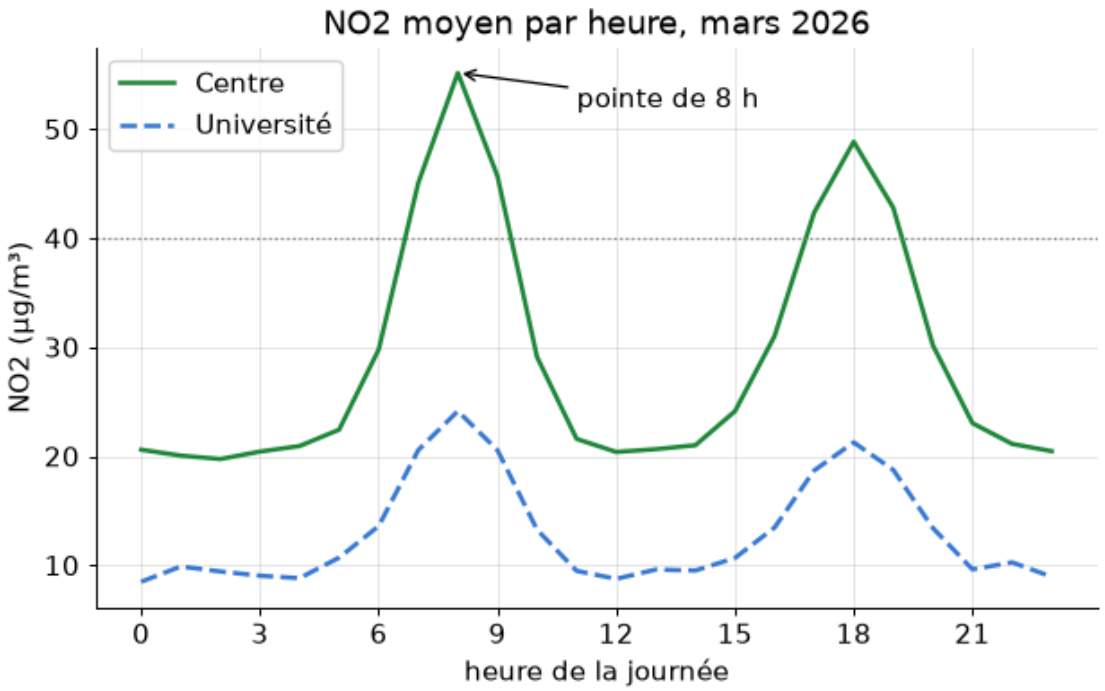
```
fig, ax = plt.subplots()
ax.plot(heures, no2)
ax.set_title("AQ-01")
ax.set_xlabel("heure")
fig.savefig("aq01.png")
```

Plus explicite, et c'est la forme qu'attendent pandas et seaborn (`ax=ax`).

PYPLOT	`plt.title`	`plt.xlabel`	`plt.xlim`	`plt.legend`
OBJET	`ax.set_title`	`ax.set_xlabel`	`ax.set_xlim`	`ax.legend`

Titre, unités, légende : un graphique qui se lit seul

```
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(p.index, p["Centre"], color="#1f883b",
        lw=2, label="Centre")
ax.plot(p.index, p["Université"], color="#3a7bd5",
        lw=2, ls="--", label="Université")
ax.axhline(40, color="gray", lw=1, ls=":")
ax.annotate("pointe de 8 h", xy=(8, p.loc[8, "Centre"]),
            xytext=(11, 52),
            arrowprops={"arrowstyle": "→"})
ax.set_title("NO2 moyen par heure, mars 2026")
ax.set_xlabel("heure de la journée")
ax.set_ylabel("NO2 (µg/m³)")
ax.set_xticks(range(0, 24, 3))
ax.grid(alpha=.3)
ax.legend()
fig.savefig("profil.png", dpi=150, bbox_inches="tight")
```



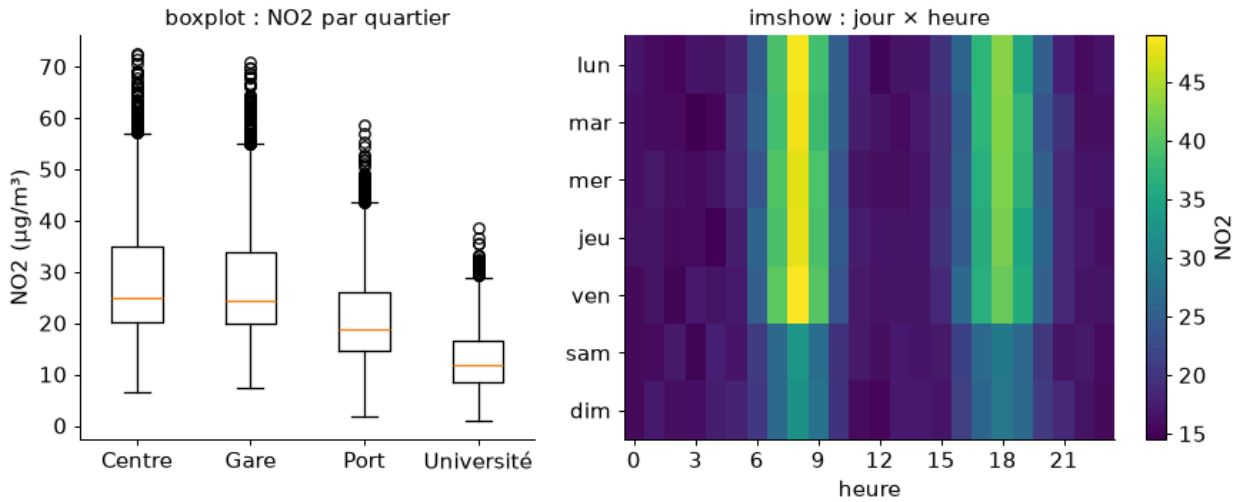
`color` `lw` `ls`	Couleur, épaisseur, style de trait
`label` + `legend`	La légende se construit seule
`dpi` `bbox_inches`	Netteté, marges rognées

Boîte à moustaches et carte de chaleur

```
fig, ax = plt.subplots(1, 2)

ax[0].boxplot([m.loc[m["quartier"] == q, "no2"]
               .dropna() for q in qs],
              tick_labels=qs)

h = m.pivot_table(index="jour", columns="heure",
                  values="no2", aggfunc="mean")
im = ax[1].imshow(h, aspect="auto",
                  cmap="viridis")
fig.colorbar(im, ax=ax[1], label="NO2")
```



À droite, le cycle journalier et le week-end, sur une seule image.

QUESTION	TRACÉ
Évolution	`plot`
Comparer des catégories	`bar`
Répartition, extrêmes	`hist`, `boxplot`
Lien entre deux mesures	`scatter`
Deux clés croisées	`imshow`

TRAVAUX PRATIQUES 4A

Traiter une image avec NumPy

≈ 50 min · Individuel · Restitution 5 min

1 Charger et inspecter

`plt.imread` sur `grace_hopper.jpg` : forme, type, valeurs extrêmes de chaque canal.

2 Passer en niveaux de gris

Par produit matriciel, puis afficher avec `cmap="gray"`.

3 Recadrer, retourner, réduire

Tranches seulement : le visage, un miroir, une image deux fois plus petite.

4 Négatif et seuillage

Quelle proportion de pixels dépasse 128 ? Afficher le masque.

5 Une planche de cinq vignettes

`plt.subplots(1, 5)`, titres, `savefig("planche.png")`.

6 Bonus

Histogramme des niveaux de gris ; éclaircir de 30 % sans déborder (`np.clip`).

Les fichiers

`tp/tp4-scipy-stack/`

`enonce-4a.md` · `depart-4a.ipynb`

Aucune donnée à copier : l'image est livrée avec Matplotlib.

Les deux pièges

Le `uint8` qui déborde, et la tranche qui modifie l'original : `.astype(float)` et `.copy()`.

SÉQUENCE 4

pandas, l'analyse de tables

Le `mesures.csv` du TP 2b : lire, filtrer, regrouper, croiser, joindre.

Series et DataFrame : des tableaux NumPy avec des noms

```python
s = pd.Series([31.0, 18.5, 44.2],
              index=["AQ-01", "AQ-07", "AQ-04"],
              name="no2")
s["AQ-04"]        # 44.2
s.mean()          # 31.23...
s.to_numpy()      # array([31. , 18.5, 44.2])
```

Series	Une colonne : des valeurs + un **index** (les étiquettes)
DataFrame	Une table : des Series qui partagent le même index

	capteur	no2	pm10
0	AQ-01	NaN	12.1
1	AQ-01	11.3	11.3
2	AQ-01	24.5	17.0

index

df["no2"] : une Series

chaque colonne a son dtype, comme un ndarray

LIRE BASE

Lire le CSV, et le regarder avant de calculer

```
m = pd.read_csv("mesures.csv",
                parse_dates=["horodatage"])
m.shape       # (8064, 6)
m.head(3)
```

```
           horodatage capteur quartier   no2  pm10  temp
0 2026-03-02 00:00:00   AQ-01   Centre   NaN  12.1   5.5
1 2026-03-02 01:00:00   AQ-01   Centre  11.3  11.3   4.4
2 2026-03-02 02:00:00   AQ-01   Centre  24.5  17.0   1.7
```

`parse_dates`

Sans lui, `horodatage` reste du texte : ni tri chronologique fiable, ni heure, ni `resample`. La deuxième question du premier matin.

```
m.info()
RangeIndex: 8064 entries, 0 to 8063
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   horodatage  8064 non-null   datetime64[ns]
 1   capteur     8064 non-null   object
 2   quartier    8064 non-null   object
 3   no2         7913 non-null   float64
 4   pm10        8064 non-null   float64
 5   temp        8064 non-null   float64
memory usage: 378.1+ KB
```

> `no2` : 7 913 valeurs sur 8 064. Il en manque 151 : on y reviendra.

SÉLECTIONNER BASE

Colonnes, lignes : `[]`, `loc`, `iloc`

```
m["no2"]                 # une colonne : Series
m[["capteur", "no2"]]    # plusieurs : DataFrame

m.loc[1, "no2"]          # 11.3   par étiquette
m.iloc[1, 3]             # 11.3   par position

m.loc[2:3, ["capteur", "no2"]]
```

```
  capteur   no2
2   AQ-01  24.5
3   AQ-01  21.4
```

	DÉSIGNE PAR	BORNE HAUTE
`loc`	**Étiquette** d'index et nom de colonne	Incluse
`iloc`	**Position**, comme NumPy	Exclue

`loc[2:3]` rend deux lignes

Avec `loc`, la tranche s'arrête **sur** l'étiquette 3. Avec `iloc[2:3]`, une seule ligne.

`m.describe()` : moyenne, écart-type, quartiles de chaque colonne numérique. NO2 moyen : 22,9 µg/m³, maximum 72,5.

FILTRER ET CHERCHER BASE

Les lignes qui répondent à une question

```
m.loc[m["no2"] > 60, ["horodatage", "capteur", "no2"]].head(3)
```

```
            horodatage capteur   no2
32  2026-03-03 08:00:00   AQ-01  61.2
392 2026-03-18 08:00:00   AQ-01  62.5
680 2026-03-02 08:00:00   AQ-02  67.9
```

```
(m["no2"] > 60).sum()                          # 82

m[(m["quartier"] == "Gare") & (m["no2"] > 60)]
m.query("quartier == 'Gare' and no2 > 60")  # 44 lignes
```

```
cap = pd.read_sql("SELECT id, nom FROM capteurs",
                  sqlite3.connect("capteurs.db"))
cap[cap["nom"].str.contains("gare|quai", case=False)]
```

```
     id                nom
3  AQ-04  Parvis de la Gare
6  AQ-07     Quai des Docks
```

Trois façons de chercher

Un **masque** pour tout ; `query` pour lire comme une phrase ; `.str` pour chercher dans du texte, avec les expressions régulières du module 2.

VALEURS MANQUANTES BASE

Compter, puis décider

```
m.isna().sum()
# horodatage 0, capteur 0, quartier 0,
# no2 151, pm10 0, temp 0

m["no2"].isna().mean()     # 0.0187 : 1,9 %

m.dropna()                 # (7913, 6)
m["no2"].fillna(m["no2"].median())
m["no2"].interpolate()     # entre voisins
# il reste 1 NaN : la toute première heure
```

Les calculs les **ignorent** par défaut : `mean()` divise par 7 913, pas par 8 064.

MÉTHODE	QUAND
`dropna`	Peu de trous, lignes indépendantes
`fillna(valeur)`	Une valeur par défaut a un sens
`interpolate`	Série temporelle régulière : **notre cas**

Jamais en silence

On compte ce qu'on retire ou remplace, et on l'écrit. La troisième question du premier matin.

NETTOYER BASE

Doublons, types, noms : la check-list d'ouverture

```python
m["capteur"].value_counts().head(3)
# AQ-01 672 · AQ-02 672 · AQ-03 672

m.duplicated(subset=["horodatage", "capteur"]).sum()
# 0 : une seule mesure par capteur et par heure

d = pd.concat([m, m.tail(2)])   # un import en double
d.duplicated().sum()            # 2
d.drop_duplicates().shape       # (8064, 6)
```

```python
m.rename(columns={"temp": "temperature"})  # une copie
m["quartier"].astype("category")
# mémoire de la colonne : 580 608 → 8 524 octets

pd.to_datetime(t, format="%d/%m/%Y %H:%M")
# texte à la française → Timestamp('2026-03-14 08:00:00')
```

```python
m.sort_values("no2", ascending=False).head(3)
```

```
             horodatage capteur   no2
1952 2026-03-27 08:00:00   AQ-03  72.5
1424 2026-03-05 08:00:00   AQ-03  72.2
1400 2026-03-04 08:00:00   AQ-03  71.5
```

VÉRIFIER	AVEC
Les catégories attendues	`value_counts`, `unique`
Les doublons	`duplicated`, `drop_duplicates`
Les types	`astype`, `to_datetime`, `to_numeric`
Les noms de colonnes	`rename`
Les extrêmes	`sort_values`, `describe`

CALCULER SI LE TEMPS LE PERMET

De nouvelles colonnes, sans boucle

```python
m["no2_ppb"] = (m["no2"] / 1.88).round(1)
m["ratio"] = m["pm10"] / m["no2"]

m["alerte"] = np.where(m["no2"] > 40,
                       "élevé", "normal")
m["alerte"].value_counts()
# normal 7211 · élevé 853

m["niveau"] = pd.cut(m["no2"], bins=[0, 20, 40, 200],
                     labels=["bas", "moyen", "élevé"])
# bas 3787 · moyen 3273 · élevé 853 · (NaN exclus)

m["nom"] = m["capteur"].map(noms)  # un dictionnaire
```

UN MILLION DE LIGNES ?	MESURÉ ICI, 8 064 LIGNES
`m["no2"] * 2`	0,08 ms
`m["no2"].apply(lambda x: x * 2)`	2,6 ms, **≈ × 30**

`apply` est une boucle déguisée

Il appelle votre fonction ligne par ligne, en Python. À réserver à ce qui n'a pas d'équivalent vectorisé.

Le piège du `NaN`

`NaN > 40` est faux : les 151 heures sans mesure sont classées « normal » par `np.where`. `pd.cut`, lui, les laisse vides.

REGROUPER BASE

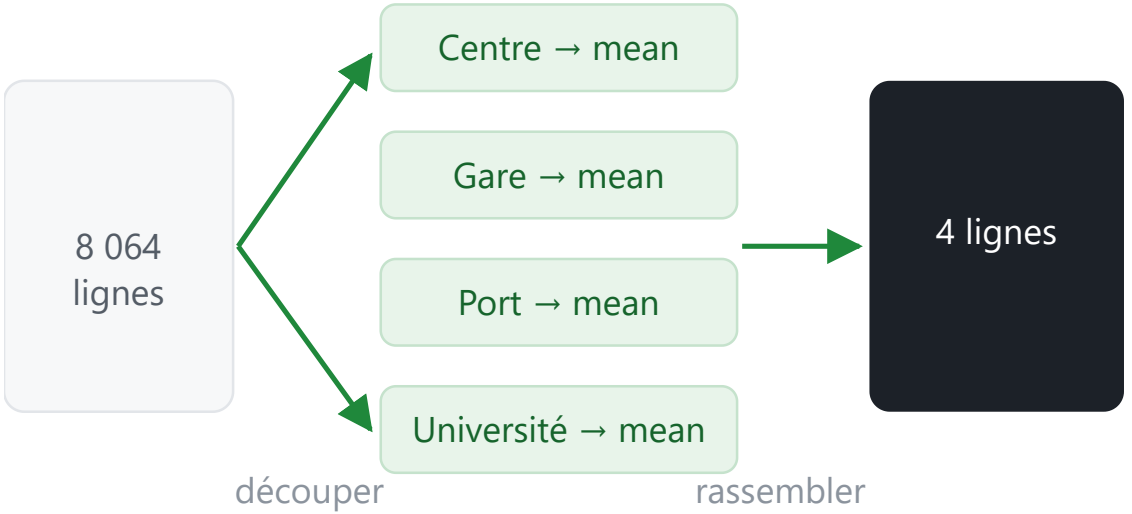
`groupby` : découper, calculer, rassembler

```
(m.groupby("quartier")["no2"].mean()
   .round(1).sort_values(ascending=False))
```

```
quartier
Centre        29.0
Gare          28.4
Port          21.4
Université    13.0
```

```
m.groupby("quartier").agg(
    no2_moy=("no2", "mean"),
    no2_max=("no2", "max"),
    n=("no2", "size")).round(1)
```

```
            no2_moy  no2_max     n
quartier
Centre         29.0     72.5  2016
Gare           28.4     70.8  2016
Port           21.4     58.6  2016
Université     13.0     38.6  2016
```



Deux clés : `m.groupby(["quartier", "capteur"])`. Le résultat a un index à deux niveaux.

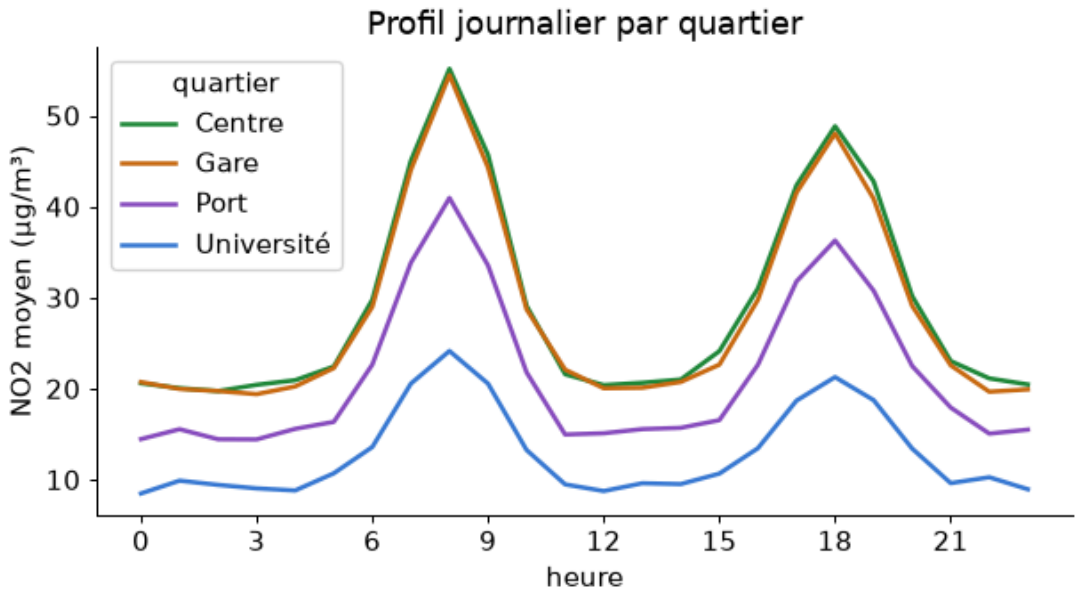
PIVOTER BASE

Le tableau croisé : heure × quartier

```
m["heure"] = m["horodatage"].dt.hour
p = m.pivot_table(index="heure",
                  columns="quartier",
                  values="no2",
                  aggfunc="mean").round(1)
p.loc[[3, 8, 13, 18]]
```

```
quartier  Centre  Gare  Port  Université
heure
3           20.4  19.4  14.4         9.1
8           55.1  54.5  41.0        24.1
13          20.7  20.1  15.6         9.6
18          48.8  48.0  36.3        21.3
```

```
p.plot()      # une courbe par colonne
```



Réponse n° 1 à la question du jour

Deux pointes, **8 h** et **18 h**, dans les quatre quartiers. Le matin est plus haut.

LE TEMPS BASE

Dates : extraire, découper, rééchantillonner

```
m["horodatage"].dt.hour         # 0 … 23
m["horodatage"].dt.dayofweek    # 0 = lundi
m["horodatage"].dt.day_name()   # 'Monday'

s = (m[m["capteur"] == "AQ-01"]
       .set_index("horodatage")["no2"])

s.loc["2026-03-03 07:00":"2026-03-03 09:00"]
# 07:00 NaN · 08:00 61.2 · 09:00 42.4

s.resample("D").mean().round(1).head(3)
# 2026-03-02 28.4 · 03-03 28.9 · 03-04 27.6
```

```
we = m["horodatage"].dt.dayofweek ≥ 5
m.groupby(["quartier", we])["no2"].mean().unstack()
```

```
horodatage  False  True
quartier
Centre       30.5   25.3
Gare         29.7   25.0
Port         22.4   18.8
Université   13.6   11.4
```

Un index de dates

Avec `set_index("horodatage")`, on découpe par texte de date et on rééchantillonne : `"D"` jour, `"W"` semaine, `"MS"` début de mois.

JOINDRE BASE

`merge` : les mesures et la base des capteurs

```
import sqlite3
cap = pd.read_sql(
    "SELECT id, nom, lat, lon FROM capteurs",
    sqlite3.connect("capteurs.db"))

t = m.merge(cap, left_on="capteur",
            right_on="id", how="left")
t.shape        # (8064, 11)

t.groupby("nom")["no2"].mean().round(1).nlargest(3)
```

```
nom
Rue Neuve               31.6
Boulevard Ferroviaire   30.3
Parvis de la Gare       28.8
```

`HOW=`	GARDE
`"inner"` (défaut)	Les lignes présentes des deux côtés
`"left"`	Toutes les mesures, même sans capteur connu
`"outer"`	Tout, des deux côtés

Vérifier le nombre de lignes

Avant 8 064, après 8 064 : la jointure n'a ni perdu ni dupliqué de mesures. Si le nombre monte, un identifiant est en double dans `cap`.

Lire et écrire Excel, et le reste

```
synthese = m.groupby("quartier").agg(
    no2_moy=("no2", "mean"),
    no2_max=("no2", "max")).round(1)

synthese.to_excel("synthese.xlsx",
                  sheet_name="NO2")

pd.read_excel("synthese.xlsx",
              sheet_name="NO2", index_col=0)
```

Il faut le paquet `openpyxl`, présent dans Anaconda.

LIRE	ÉCRIRE	POUR
`read_csv`	`to_csv`	Échanger, archiver en texte
`read_excel`	`to_excel`	Remettre à un métier
`read_sql`	`to_sql`	Une base relationnelle
`read_parquet`	`to_parquet`	Gros volumes, types conservés

CSV français

`read_csv(f, sep=";", decimal=",")` pour un export Excel en français.

ASSEMBLER ET TRACER · SI LE TEMPS LE PERMET

Deux cents fichiers, un tableau, un graphique

Assembler avec `glob` et `concat`

```python
import glob

fichiers = sorted(glob.glob("exports/mesures-*.csv"))
# ['exports/mesures-Centre.csv', 'exports/mesures-Gare.csv',
#  'exports/mesures-Port.csv', 'exports/mesures-Université.csv']

tout = pd.concat(
    [pd.read_csv(f, parse_dates=["horodatage"])
     for f in fichiers],
    ignore_index=True)
tout.shape     # (8064, 6)
```

Tracer directement depuis pandas

```python
moy = m.groupby("quartier")["no2"].mean()
moy.plot.barh()                # des barres
m["no2"].plot.hist(bins=40)    # un histogramme
p.plot(ax=ax)                  # dans un axes existant
```

`.plot()` / `.plot.line`	Courbes, une par colonne
`.plot.bar` `.barh`	Barres, l'index en étiquettes
`.plot.hist` `.box`	Distributions
`.plot.scatter(x=, y=)`	Nuage de deux colonnes

> Chaque appel rend un **axes Matplotlib** : on l'habille avec `set_title`, `set_ylabel`, comme ce matin.

TRAVAUX PRATIQUES 4B

Les statistiques d'Air'Cité

≈ 45 min · Individuel · Restitution 5 min

1. **Lire et contrôler**

   `mesures.csv` avec `parse_dates` ; `info`, trous par capteur.

2. **Interpoler capteur par capteur**

   Compter avant et après ; écrire le nombre remplacé.

3. **Classer**

   NO2 moyen et maximum par quartier, puis par capteur avec son nom (jointure SQLite).

4. **Le profil journalier**

   Pivot heure × quartier, tracé avec titres et unités, enregistré en PNG.

5. **Semaine et week-end**

   Le tableau quartier × type de jour ; en barres groupées.

6. **Bonus 1**

   Moyennes journalières par capteur, exportées en Excel, un onglet par quartier.

7. **Bonus 2**

   Les mêmes questions en SQL avec DuckDB, sur le CSV puis sur le DataFrame.

Les fichiers

`tp/tp4-scipy-stack/`

`enonce-4b.md` · `depart-4b.ipynb`

Vos `mesures.csv` et `capteurs.db` du TP 2b, ou la copie de référence dans `tp/donnees/`.

Le livrable

Deux figures (`profil.png`, `semaine.png`) et trois chiffres à annoncer : le quartier le plus exposé, l'heure de pointe, la baisse du week-end.

SÉQUENCE 5

SciPy, les outils du scientifique

Tester, mesurer des distances, chercher un rythme.

PANORAMA BASE

SciPy : une boîte de sous-modules

SOUS-MODULE	POUR
`scipy.stats`	Lois, statistiques descriptives, tests
`scipy.spatial`	Distances, plus proches voisins, enveloppes
`scipy.fft`	Transformée de Fourier
`scipy.signal`	Filtrage, détection de pics
`scipy.interpolate`	Courbes entre des points mesurés
`scipy.optimize`	Minimiser, ajuster un modèle
`scipy.ndimage`	Filtres sur images
`scipy.linalg`	Algèbre linéaire étendue

Même contrat que NumPy

Des tableaux en entrée, des tableaux en sortie. Une colonne pandas s'y passe telle quelle.

Trois aujourd'hui

`stats`, `spatial`, `fft` : les réponses à la question du jour. `interpolate` en complément, `optimize` et `signal` en annexe.

TESTER BASE

La baisse du week-end est-elle due au hasard ?

```python
from scipy import stats

we = m["horodatage"].dt.dayofweek ≥ 5
semaine = m.loc[~we, "no2"].dropna()
weekend = m.loc[we, "no2"].dropna()
semaine.mean(), weekend.mean()   # 24.1, 20.1

r = stats.ttest_ind(semaine, weekend,
                    equal_var=False)
print(f"t = {r.statistic:.1f}   p = {r.pvalue:.1e}")
# t = 15.4   p = 9.2e-53
```

Lire la p-valeur

La probabilité d'observer un tel écart **si les deux périodes étaient identiques**. Ici, 10^{-52} : le hasard n'explique pas la baisse.

Contre-exemple

Mardi 3 contre mercredi 4 mars : **p = 0,84**. Aucune différence démontrée, et c'est la bonne réponse.

`stats.describe(m["no2"].dropna())` : 7 913 valeurs, moyenne 22,94, variance 151, asymétrie 1,09.

CORRÉLATION BASE

Deux mesures qui montent ensemble

```
ok = m.dropna(subset=["no2"])

stats.pearsonr(ok["no2"], ok["pm10"]).statistic
# 0.88
stats.pearsonr(ok["no2"], ok["temp"]).statistic
# 0.13

ok[["no2", "pm10", "temp"]].corr().round(2)
```

```
       no2  pm10  temp
no2   1.00  0.88  0.13
pm10  0.88  1.00  0.12
temp  0.13  0.12  1.00
```

COEFFICIENT R	LECTURE
proche de 1	Montent ensemble, presque en ligne droite
proche de 0	Pas de lien **linéaire**
proche de −1	L'un monte, l'autre descend

Corrélation n'est pas cause

NO2 et PM10 ne se causent pas l'un l'autre : **le trafic** produit les deux.

La loi normale, et ce qu'elle oublie

```
v = m["no2"].dropna()
n = stats.norm(loc=v.mean(), scale=v.std())
# moyenne 22.9, écart-type 12.3

n.sf(40)          # 0.083 : P(NO2 > 40) selon le modèle
(v > 40).mean()   # 0.108 : la réalité

n.ppf(0.95)       # 43.2 : 95e centile du modèle
v.quantile(0.95)  # 48.8 : 95e centile réel
```

MÉTHODE	DONNE
`pdf(x)`	La densité, la hauteur de la cloche
`cdf(x)`	P(X ≤ x)
`sf(x)`	P(X > x), soit `1 - cdf(x)`
`ppf(q)`	Le seuil sous lequel tombe la proportion q
`rvs(n)`	n tirages au hasard

Le modèle sous-estime les pointes

La distribution est asymétrique (1,09). Une cloche symétrique prévoit 8 % d'heures au-dessus de 40 ; il y en a 11 %.

GÉOSPATIAL BASE

Quel capteur est le plus proche de l'école ?

```python
from scipy.spatial import KDTree

# degrés → kilomètres, approximation locale
k = 111.32 * np.cos(np.radians(45.765))
xy = np.c_[cap["lon"] * k, cap["lat"] * 110.57]

arbre = KDTree(xy)
ecole = [4.845 * k, 45.762 * 110.57]
dist, i = arbre.query(ecole, k=2)

dist.round(2)            # array([0.65, 0.76])
cap.loc[i, "nom"].tolist()
# ['Place des Halles', 'Hôtel de Ville']
```

Un arbre pour chercher vite

`KDTree` range les points une fois ; chaque recherche des *k* plus proches voisins est ensuite quasi instantanée, même sur des millions de points.

Des degrés ne sont pas des kilomètres

Un degré de longitude vaut 111 km à l'équateur, 78 km à cette latitude. On convertit avant de mesurer.

`distance.cdist(xy, xy)` : la matrice 12 × 12 de toutes les distances. Les deux capteurs les plus éloignés : 5,05 km.

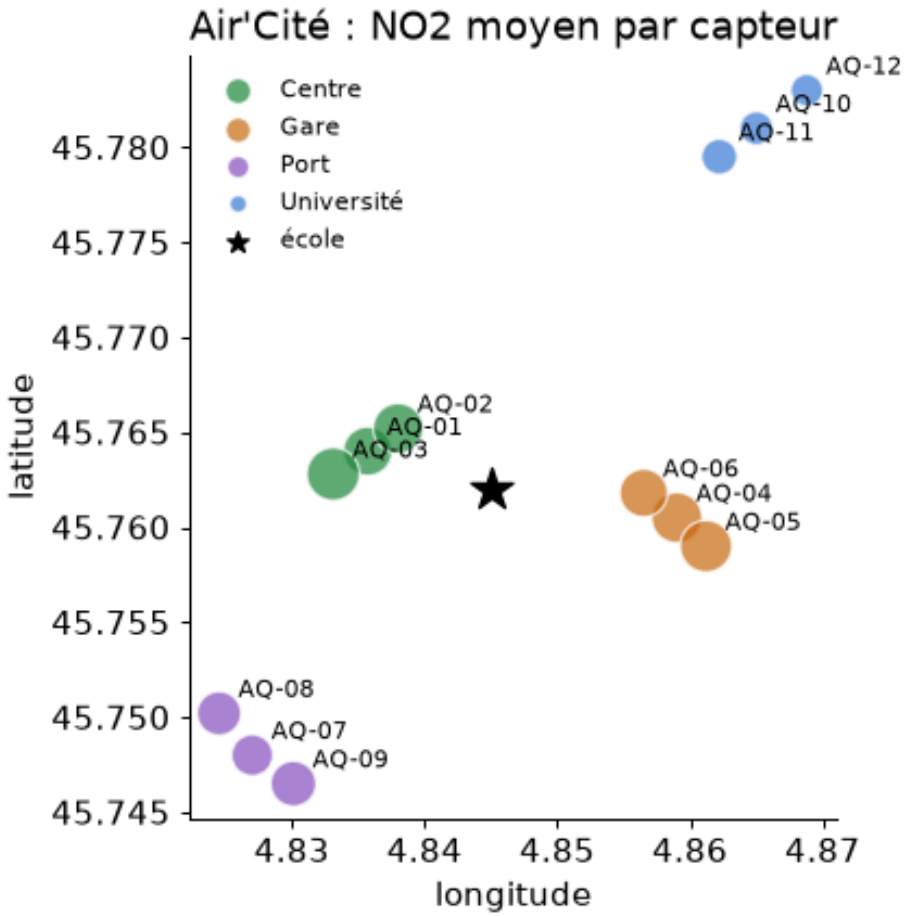
CARTOGRAPHIE BASE

Une première carte, avec Matplotlib seul

```
fig, ax = plt.subplots()
for q, g in cap.groupby("quartier"):
    ax.scatter(g["lon"], g["lat"],
               s=g["no2_moy"] * 12,
               label=q, alpha=.7)
ax.scatter([4.845], [45.762],
           marker="*", s=260, c="black",
           label="école")
ax.set_aspect(1 / np.cos(np.radians(45.765)))
ax.legend()
```

Pour aller plus loin

geopandas (formes, projections), **folium** (carte web interactive), **cartopy** (fonds et projections).



Taille : NO2 moyen sur quatre semaines. Étoile : l'école.

INTERPOLER SI LE TEMPS LE PERMET

Combler un trou : droite ou courbe ?

```python
from scipy import interpolate

s = (m[m["capteur"] == "AQ-01"]
       .set_index("horodatage")["no2"])
s.loc["2026-03-03 05:00":"2026-03-03 10:00"].tolist()
# [16.1, 34.2, nan, 61.2, 42.4, 31.6]

h = np.arange(len(s))        # heures écoulées
ok = s.notna().to_numpy()
lin = interpolate.interp1d(h[ok], s[ok])
cub = interpolate.CubicSpline(h[ok], s[ok])

lin(31), cub(31)   # 3 mars, 7 h : 47.7 et 56.7
```

MÉTHODE	ENTRE 34,2 ET 61,2
`interp1d` : une droite entre voisins	47,7
`CubicSpline` : une courbe lisse	56,7
`s.interpolate()` de pandas	47,7, la droite

Le trou tombe sur la montée de 8 h

La droite coupe le virage, la courbe le suit. Aucune n'est « la vraie valeur » : c'est un choix, à écrire.

FOURIER BASE

Décomposer un signal en rythmes

Toute série régulière s'écrit comme une **somme de sinusoïdes**. La transformée de Fourier dit lesquelles, et avec quelle force.

```
from scipy import fft

s = m.groupby("horodatage")["no2"].mean()  # la ville
y = s.to_numpy() - s.mean()      # centrer
len(y)                           # 672 heures

spectre = np.abs(fft.rfft(y))
freqs = fft.rfftfreq(len(y), d=1)   # d = 1 heure
periodes = 1 / freqs[1:]            # en heures
```

`rfft`	Transformée d'un signal réel : un nombre complexe par fréquence
`np.abs`	L'amplitude : la « force » de chaque rythme
`rfftfreq(n, d)`	Les fréquences correspondantes, en cycles par `d`
`1 / f`	La période, plus parlante : 24 h, 168 h…

Les conditions

Un pas **régulier** et **sans trou** : d'où la moyenne sur la ville, ou l'interpolation du TP 4b pour un capteur seul.

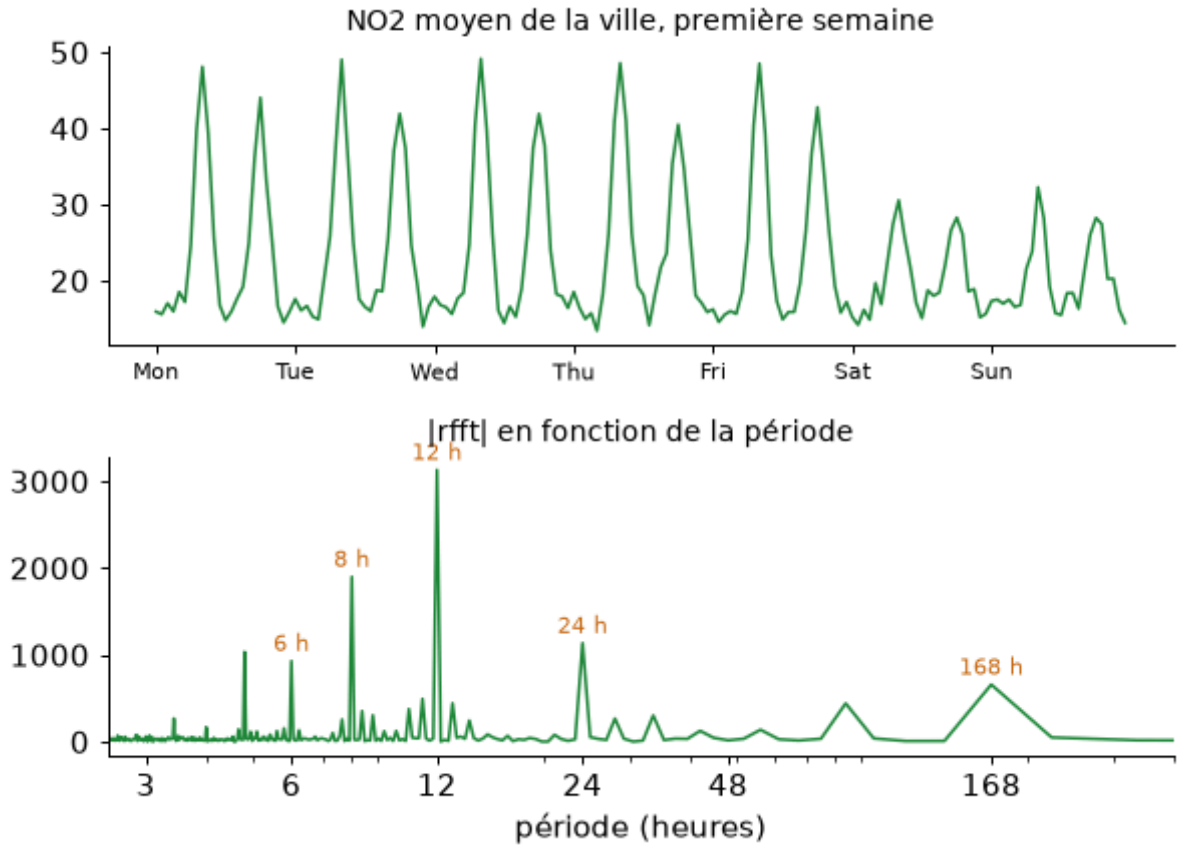
Le spectre d'Air'Cité : une famille de pics

```
top = np.argsort(spectre)[::-1][:6]
for j in top:
    print(f"{1/freqs[j]:6.1f} h  {spectre[j]:5.0f}")
```

```
  12.0 h   3133
   8.0 h   1901
  24.0 h   1138
   4.8 h   1036
   6.0 h    932
 168.0 h    660
```

Réponse n° 3 à la question du jour

Toutes les périodes divisent **24 h** : c'est le cycle journalier, avec ses harmoniques. Et **168 h**, c'est la semaine.



Axe des périodes en échelle logarithmique.

TRAVAUX PRATIQUES 4C

La carte et le spectre

≈ 35 min · Individuel · Restitution 5 min

1. **Tester le week-end**

   Test t de Welch, puis le même test quartier par quartier : la baisse est-elle partout significative ?

2. **La carte**

   Capteurs en lon/lat, taille = NO2 moyen, couleur = quartier, `set_aspect` corrigé.

3. **Le plus proche**

   Pour trois adresses fournies dans l'énoncé, le capteur le plus proche et sa distance en km.

4. **Le spectre**

   Série de la ville, `rfft`, les cinq périodes dominantes, tracé en fonction de la période.

5. **Bonus**

   Le spectre d'**un** capteur après interpolation ; et celui de la température : combien de pics ?

Les fichiers

`tp/tp4-scipy-stack/`

`enonce-4c.md` · `depart-4c.ipynb`

Repartir du DataFrame interpolé du TP 4b.

Le livrable

`carte.png`, `spectre.png`, et une phrase : « le NO2 d'Air'Cité suit … ».

CONTRÔLE DES ACQUIS

Cinq questions avant le bilan

Q1 Vous écrivez une boucle `for` sur un tableau d'un million de valeurs. Que cherchez-vous à la place ?

L'opération vectorisée qui fait la même chose d'un coup : un opérateur, une fonction NumPy, un masque. Elle s'exécute en C, des dizaines de fois plus vite.

Q2 `m.mean(axis=0)` sur un tableau de 3 lignes et 4 colonnes : quelle forme a le résultat ?

`(4,)` : l'axe 0, les lignes, disparaît ; il reste une moyenne par colonne.

Q3 Après un `merge`, le DataFrame a plus de lignes qu'avant. Que s'est-il passé ?

La clé de jointure est en double dans la table de droite : chaque mesure a été recopiée autant de fois. On vérifie toujours le nombre de lignes après une jointure.

Q4 Une p-valeur de 0,84 sur la comparaison de deux jours : conclusion ?

Aucune différence démontrée entre ces deux jours. Ce n'est pas une preuve qu'ils sont identiques, seulement l'absence de preuve qu'ils diffèrent.

Q5 Le spectre du NO2 a son plus grand pic à 12 h, pas à 24 h. Le cycle est-il de 12 h ?

Non : le cycle est journalier, mais avec deux pointes. Sa forme se décompose en 24 h et ses harmoniques, dont 12 h est la plus forte.

CONTRÔLE DES ACQUIS

Le quiz en ligne du module 4

Il porte sur ce que le module vient de couvrir : NumPy, Matplotlib, pandas et SciPy.

```
wytasoft.com/quizz/python-traitement-donnees-module-4-fr
```

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

ANNEXES DU MODULE 4 AVANCÉ

Ajuster et détecter

Ajuster un modèle à des mesures, trouver des pics sans les chercher à l'œil.

SCIPY.OPTIMIZE AVANCÉ

Ajuster une sinusoïde à la température

```
from scipy import optimize

t = m.groupby("heure")["temp"].mean()

def modele(h, moy, amp, decal):
    return moy + amp * np.sin(
        2 * np.pi * (h - decal) / 24)

p, _ = optimize.curve_fit(
    modele, t.index, t.values,
    p0=[10, 5, 9])
p.round(2)     # array([10.99,  5.01,  9.02])
```

Ce que disent les trois nombres

Moyenne 11 °C, amplitude ± 5 °C, maximum vers **15 h** (décalage 9 h + un quart de période).

`p0`, le point de départ

L'ajustement part d'une estimation et l'améliore. Une estimation absurde peut le faire échouer : on donne des ordres de grandeur plausibles.

SCIPY.SIGNAL AVANCÉ

Trouver les pointes sans les chercher à l'œil

```
from scipy import signal

prof = m.groupby("heure")["no2"].mean()
pics, _ = signal.find_peaks(prof, prominence=5)

pics                          # array([ 8, 18])
prof.iloc[pics].round(1).tolist()   # [43.7, 38.5]
```

La proéminence

De combien le pic dépasse ce qui l'entoure. Sans elle, chaque petite oscillation du bruit serait un « pic ».

Sur une série longue, la même ligne repère les épisodes de pollution, jour après jour, pour les compter ou les dater.

CLÔTURE

Bilan des trois jours

Ce qui a été couvert, les questions du premier matin, et la suite.

BILAN

Le programme, et ce qui a été produit

OBJECTIF DU PROGRAMME	OÙ	CE QUI A ÉTÉ PRODUIT
Savoir programmer avec le langage Python	Modules 1 et 2	Les exercices des TP 1a et 1b ; la classe du TP 2a ; le journal analysé, la base SQLite et `mesures.csv` du TP 2b
Une vue d'ensemble de l'écosystème scientifique	Module 3	L'environnement `aircite` exporté et dupliqué (TP 3a) ; un notebook qui compare CSV, Parquet, Excel et JSON (TP 3b)
Les librairies scientifiques incontournables	Module 4	Une planche d'images, des statistiques, un profil, une carte et un spectre

> Un seul fil : **du journal brut du TP 2b au spectre du TP 4c**. Chaque module a produit ce que le suivant a utilisé.

BILAN

Les trois questions du premier matin

Q1 Additionner une colonne dans 200 fichiers Excel : quelle est la partie difficile ?

La répétition, qu'une boucle et une fonction automatisent. Avec pandas : une boucle sur les fichiers, `read_excel`, `pd.concat`, puis une seule somme.

Q2 `2026-03-14T08:00:00` : un texte ou une date ?

Un texte, tant qu'on ne l'a pas converti. `parse_dates` en a fait une date ce matin : c'est elle qui a permis `.dt.hour`, `resample`, le week-end et Fourier.

Q3 0,5 % de lignes abîmées : les ignorer, les corriger, ou tout arrêter ?

Jamais en silence. Au TP 2b, les lignes rejetées du journal ont été comptées ; aujourd'hui, les 151 trous du NO2 ont été comptés, puis interpolés, et le nombre écrit.

POUR CONTINUER

Après la formation

Les bibliothèques suivantes

scikit-learn	L'apprentissage automatique : prédire, classer, regrouper. Il prend des tableaux NumPy et des DataFrame.
seaborn	Des graphiques statistiques en une ligne, bâtis sur Matplotlib.
Polars	Des DataFrame plus rapides que pandas sur de gros fichiers, avec une syntaxe différente.
geopandas, **folium**	De vraies cartes, avec contours et fonds de plan.

Les références

- **numpy.org**, **pandas.pydata.org**, **matplotlib.org**, **scipy.org** : guides d'utilisation et références, avec des exemples.
- La galerie de Matplotlib : partir d'un graphique qui ressemble au sien, et copier son code.

Le premier pas lundi

Prendre un fichier que vous traitez à la main chaque semaine, et le traiter dans un notebook. C'est l'exercice le plus rentable.

CLÔTURE

Évaluation et attestation

1. **L'évaluation de la formation**

   Le questionnaire de l'organisme, à remplir maintenant, avant de partir. Dix minutes.

2. **La feuille de présence**

   Signée pour la dernière demi-journée.

3. **L'attestation de fin de formation**

   Transmise par l'organisme.

4. **Le support**

   Tous les decks en PDF, avec le dossier `tp/` et ses corrigés.

Vos retours servent vraiment

Ce qui a été trop long, trop court, trop rapide : c'est ce qui change la session suivante. Une phrase précise vaut mieux qu'une note.

Merci

Questions, retours, et bon traitement de données avec Python.

Mehdi Tajmouati · Support de cours transmis en fin de session.

✉️ mehdi.tajmouati@wytasoft.com 🌐 www.wytasoft.com 💼 LinkedIn 📚 Udemy ✍️ Medium